



Modelo Numérico da Atmosfera

MET-576-4

Modelagem Numérica da Atmosfera

Dr. Paulo Yoshio Kubota

Os métodos numéricos, formulação e parametrizações utilizados nos modelos atmosféricos serão descritos em detalhe.

**3 Meses
24 Aulas (2 horas cada)**



Dinâmica:

Métodos numéricos amplamente utilizados na solução numérica das equações diferenciais parciais que governam os movimentos na atmosfera serão o foco, mas também serão analisados os novos conceitos e novos métodos.



- ✓ **Métodos de diferenças finitas.**
- ✓ **Acurácia.**
- ✓ **Consistência.**
- ✓ **Estabilidade.**
- ✓ **Convergência.**
- ✓ **Grades de Arakawa A, B, C e E.**
- ✓ **Domínio de influência e domínio de dependência.**
- ✓ **Dispersão numérica e dissipação.**
- ✓ **Definição de filtros monótono e positivo.**
- ✓ **Métodos espectrais.**
- ✓ **Métodos de volume finito.**
- ✓ **Métodos Semi-Lagrangeanos.**
- ✓ **Conservação de massa local.**
- ✓ **Esquemas explícitos versus semi-implícitos.**
- ✓ **Métodos semi-implícitos.**



Consistência

O FDE é **consistente** com o PDE se, no limite $\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, o FDE coincidir com o PDE. Obviamente, esse é um requisito básico que o FDE deve atender se suas soluções forem boas aproximações das soluções do PDE.

A consistência é bastante simples de verificar:

- Substitua u em vez de U no FDE.
- Avalie todos os termos usando uma expansão da série de Taylor centrada no ponto $(x_j t_n)$.
- Subtraia o PDE do FDE.

Se a diferença (erro de truncamento local) for para zero como $\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, então o FDE é consistente com o PDE.



Consistência

DEFINIÇÃO

A (Equação de Diferenças Finitas) FDE é **consistente** com a (Equação Diferencial Parcial) PDE se, no limite $\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, o FDE coincidir com o PDE.

Este é um requisito fundamental: garante que a solução numérica seja uma boa aproximação da solução analítica.

Como verificar a consistência:

- 1 Substituição**
Substituir u (solução exata) em vez de U (solução numérica) no FDE.
- 2 Expansão de Taylor**
Avaliar todos os termos usando expansão em série de Taylor centrada em (x_j, t_n) .
- 3 Subtração**
Subtrair o PDE do FDE — o resultado é o erro de truncamento local.

■ **CRITÉRIO:** Se o erro de truncamento local $\rightarrow 0$ quando $\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, então o FDE é consistente com o PDE.



Consistência — Exemplos Práticos em Meteorologia



Equação de Advecção Linear | FDE Upstream vs FDE Centrado no Tempo

✓ CASO CONSISTENTE

Esquema Upstream (Forward-Time / Backward-Space)

Contexto meteorológico:

Advecção de temperatura potencial θ pelo vento u em um modelo de previsão numérica do tempo (NWP).

$$\frac{\theta_j^{n+1} - \theta_j^n}{\Delta t} + u \frac{\theta_j^n - \theta_{j-1}^n}{\Delta x} = 0$$

Erro de truncamento local:

$$\tau = O(\Delta t) + O(\Delta x) \rightarrow 0 \quad \text{quando} \quad \Delta t, \Delta x \rightarrow 0$$

✓ **Consistente:** o erro vai a zero, com o refinamento da grade — o FDE reproduz corretamente a física da advecção.

✗ CASO NÃO CONSISTENTE

Esquema com Erro de Discretização Residual

Contexto meteorológico:

Tentativa de discretizar a mesma equação de advecção de θ , mas com um termo de correção incorreto introduzido por engano.

$$\frac{\theta_j^{n+1} - \theta_j^n}{\Delta t} + u \frac{\theta_j^n - \theta_{j-1}^n}{\Delta x} + C = 0$$

Erro de truncamento local:

$$\tau = O(\Delta t) + O(\Delta x) + C \rightarrow C \neq 0 \quad \text{quando} \quad \Delta t, \Delta x \rightarrow 0$$

✗ **Não consistente:** o termo C (ex.: parametrização de difusão aplicada incorretamente) não desaparece — o FDE nunca converge para o PDE real.

Implicação prática: um modelo não consistente acumula erros físicos espúrios — mesmo com Δt e Δx pequenos, a previsão diverge da atmosfera real.



Erro de truncamento e Consistência

Vamos verificar a consistência do **esquema upstream** para a equação de advecção. Primeiro, considere a expansão da série Taylor:

$$\left\{ \begin{array}{l} u_j^{n+1} = \left(u + u_t \Delta t + \frac{1}{2} u_{tt} \Delta t^2 + \dots \right)_j^n \\ u_{j-1}^n = \left(u - u_x \Delta x + \frac{1}{2} u_{xx} \Delta x^2 - \dots \right)_j^n \end{array} \right\} \quad \left\{ \begin{array}{l} u_j^{n+1} - u_j^n = \left(u_t \Delta t + \frac{1}{2} u_{tt} \Delta t^2 + \dots \right)_j^n \\ u_{j-1}^n - u_j^n = \left(-u_x \Delta x + \frac{1}{2} u_{xx} \Delta x^2 + \dots \right)_j^n \end{array} \right. \quad \left\{ \begin{array}{l} \frac{u_j^{n+1} - u_j^n}{\Delta t} = \left(u_t + \frac{1}{2} u_{tt} \Delta t + \dots \right)_j^n \\ \frac{u_{j-1}^n - u_j^n}{\Delta x} = \left(-u_x + \frac{1}{2} u_{xx} \Delta x + \dots \right)_j^n \end{array} \right.$$

Nós substituímos isso no FDE e obtemos

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} + c \frac{U_j^n - U_{j-1}^n}{\Delta x} = 0$$

$$\left(u_t + \frac{1}{2} u_{tt} \Delta t + \dots \right)_j^n + c \left(u_x - \frac{1}{2} u_{xx} \Delta x + \dots \right)_j^n \simeq 0$$

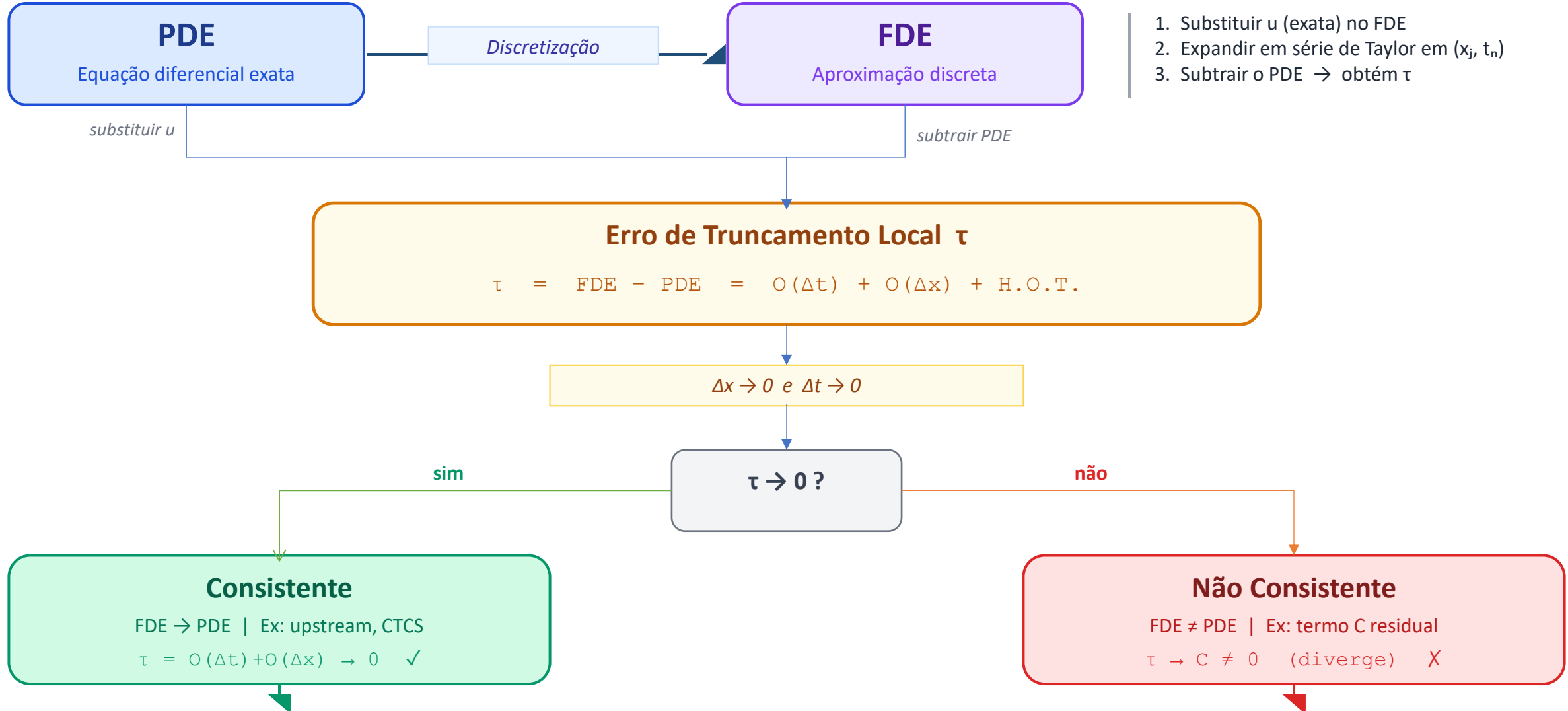
Subtrair o PDE resulta no erro de truncamento local:

$$\tau = \left(\frac{u_{tt}}{2} \right) \Delta t - \left(\frac{cu_{xx}}{2} \right) \Delta x + \text{H.O.T.} = O(\Delta t) + O(\Delta x)$$



Consistência e Erro de Truncamento

A relação entre FDE, PDE e o limite do erro de truncamento



Teorema de Lax-Richtmyer: Consistência + Estabilidade $\Rightarrow$ Convergência



Erro de truncamento e Consistência

Novamente, o erro de truncamento local é:

$$\tau = \left(\frac{u_{tt}}{2}\right) \Delta t - \left(\frac{cu_{xx}}{2}\right) \Delta x + \text{H.O.T.} = O(\Delta t) + O(\Delta x)$$

Claramente, quando $\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, temos $\tau \rightarrow 0$. Portanto, o FDE é consistente.

Observe que os erros **de truncamento de tempo e espaço** são **de primeira ordem**, porque as diferenças finitas **não estão centradas** tanto no espaço quanto no tempo.

Erros de truncamento para **diferenças centralizadas** são de **segunda ordem**.

Portanto, em geral, diferenças centralizadas são mais precisas do que diferenças não centradas.

###

Erros de truncamento são um fator crucial para determinar a precisão da previsão no NWP.



Convergência e Estabilidade

Questão central: a solução do FDE converge para a solução do PDE quando $\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$?

Resposta: a convergência pode ser garantida analisando a estabilidade computacional do esquema.

Equação de Advecção Linear

Equação governante:

$$\partial u / \partial t + c \cdot \partial u / \partial x = 0$$

Solução analítica:

$$u(x, t) = u(x + ct, 0)$$

Interpretação Física

- **Translação:** A forma inicial $u(x, t=0)$ propaga-se ao longo do eixo x sem deformação.
- **Velocidade:** A propagação ocorre com velocidade constante c .
- **Conservação:** A forma é preservada — sem dissipação ou dispersão.
- **Objetivo NWP:** O esquema numérico deve reproduzir este comportamento.

Nota: $U(j\Delta x, n\Delta t) \rightarrow u(x, t)$ quando $j\Delta x \rightarrow x$ e $n\Delta t \rightarrow t$



Convergência e Estabilidade

A segunda questão levantada acima foi se a solução do FDE converge para a solução do PDE.

Isto é, se deixarmos $\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, de modo que $j\Delta x \rightarrow x$ e $n\Delta t \rightarrow t$, portanto, $U(j\Delta x, n\Delta t) \Rightarrow u(x, t)$?

Isso é claramente importante e pode ser respondido considerando outro problema, o da estabilidade computacional.

Considere novamente a equação de advecção

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0,$$

que tem a solução $u(x, t) = u(x + ct, 0)$

A forma da solução $u(x, 0)$ se translada ao longo do eixo x com velocidade c (veja a Figura abaixo).

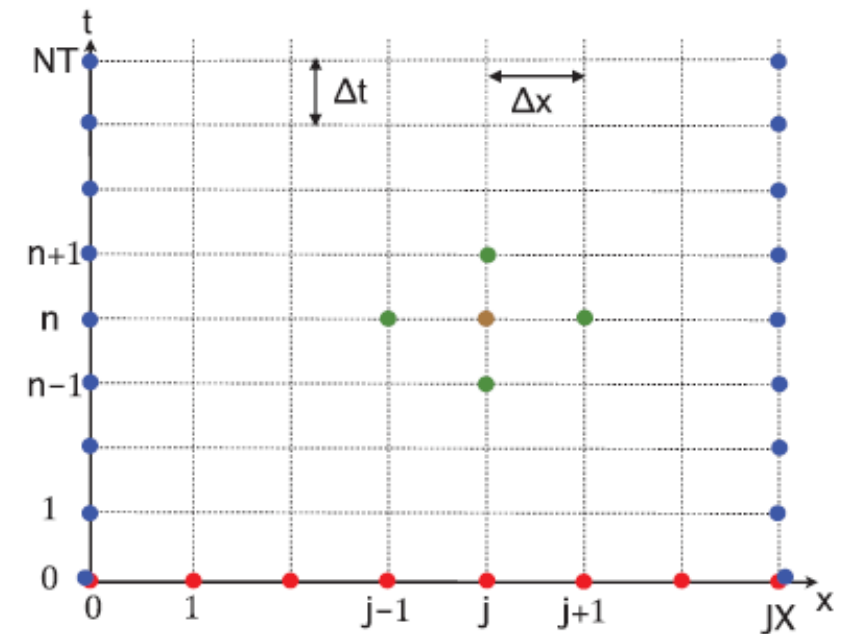
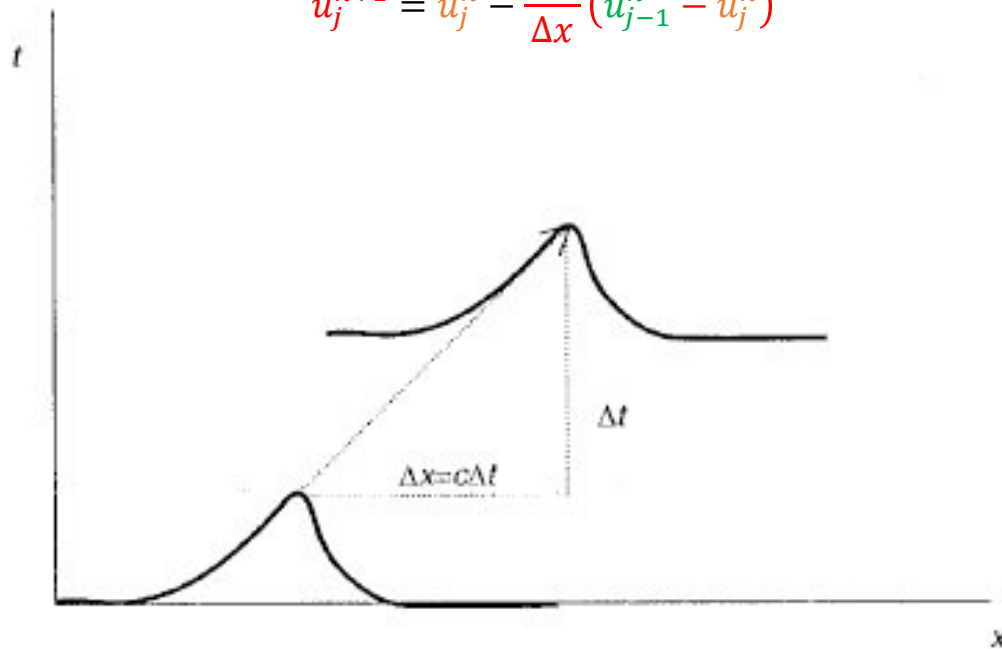


Convergência e Estabilidade

$\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, de modo que $j\Delta x \rightarrow x$ e $n\Delta t \rightarrow t$, portanto, $U(j\Delta x, n\Delta t) \Rightarrow u(x, t)$?

A forma da solução $u(x, 0)$ se translada ao longo do eixo x com velocidade c (veja a Figura abaixo).

$$u_j^{n+1} = u_j^n - \frac{c\Delta t}{\Delta x} (u_{j-1}^n - u_j^n)$$



"Figura 4.1. Uma grade de diferenças finitas no tempo e no espaço. Os pontos da grade utilizados pelos esquemas centrados tanto no tempo quanto no espaço são mostrados como pontos verdes. Os pontos vermelhos são os pontos necessários para a condição inicial e os pontos azuis ilustram as duas fronteiras, nas quais as condições devem ser prescritas. O ponto marrom mostra o ponto 'aqui e agora' j, n ."

Esquema da **solução** da equação de advecção (para $c > 0$).



Convergência e Estabilidade

$\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, de modo que $j\Delta x \rightarrow x$ e $n\Delta t \rightarrow t$, portanto, $U(j\Delta x, n\Delta t) \Rightarrow u(x, t)$?

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0,$$

O FDE para o esquema upstream pode ser escrito como

$$U_j^{n+1} = (1 - \mu)U_j^n + \mu U_{j-1}^n$$

onde

$$\mu \equiv \frac{c\Delta t}{\Delta x}$$

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = -c \frac{u_{j-1}^n - u_j^n}{\Delta x}$$

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} + c \frac{U_j^n - U_{j-1}^n}{\Delta x} = 0$$

$$u_j^{n+1} - u_j^n = -\frac{c\Delta t}{\Delta x} (u_{j-1}^n - u_j^n)$$

μ é o número Courant (ou número Lewy).

Vamos supor que $0 \leq \mu \leq 1$.

Então a solução FDE no novo nível de tempo U_j^{n+1} é **interpolada** entre os valores U_j^n e U_{j-1}^n .

Nesse caso, o esquema de advecção funciona da maneira que deveria, porque a verdadeira solução está entre esses valores (U_j^{n+1} e U_j^n).



Modelo Numérico da Atmosfera

Convergência e Estabilidade — Interpretação das Figuras

Figura 1 — grade espaço-tempo (upstream)

Para calcular x_j no tempo t_{n+1} , o esquema usa os dois pontos vizinhos em t_n . A área verde é o domínio de influência — a região que contribui para o cálculo.

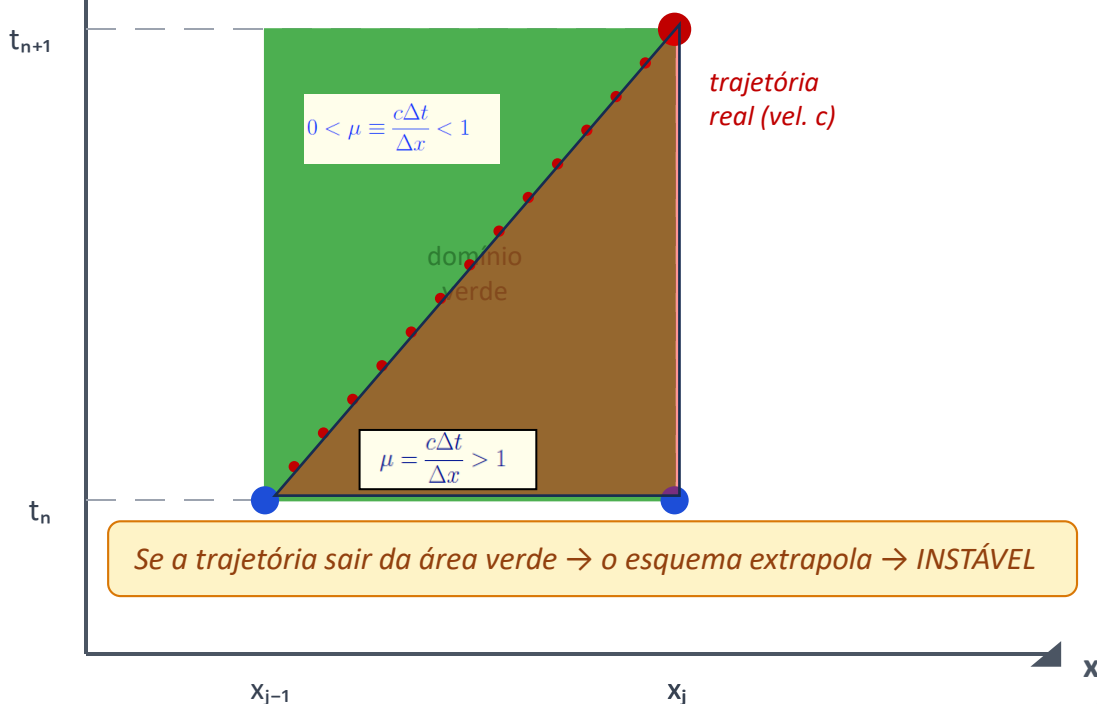


Figura 2 — condição CFL de estabilidade

$$\mu = c\Delta t / \Delta x \leftarrow \text{número de Courant}$$

μ controla se a trajetória real fica dentro do domínio de influência.

ESTÁVEL

$$\mu \leq 1$$

$$c \cdot \Delta t \leq \Delta x$$

interpolação ✓

INSTÁVEL

$$\mu > 1$$

$$c \cdot \Delta t > \Delta x$$

extrapolação ✗

Esquema upstream (interpolação linear):

$$U_j^{n+1} = (1-\mu) \cdot U_j^n + \mu \cdot U_{j-1}^n$$

peso $(1-\mu)$
se $\mu \in [0,1]$
 ≥ 0 (ok)

e

peso μ
se $\mu > 1$
 $< 0 \rightarrow$ instável!



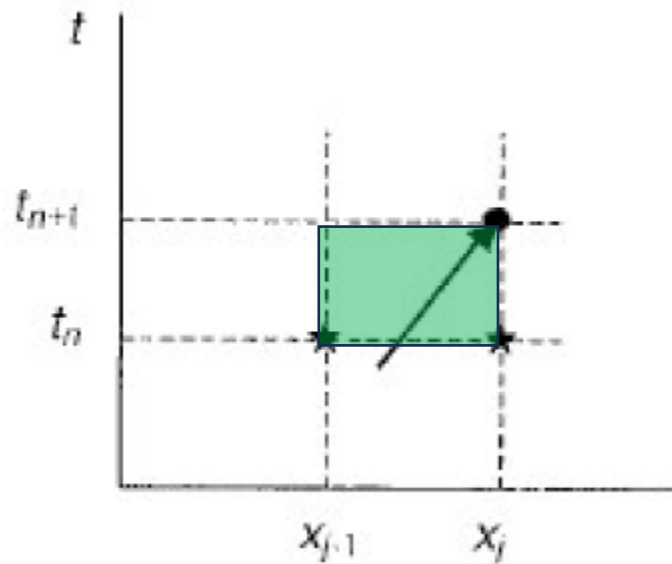
Modelo Numérico da Atmosfera



Convergência e Estabilidade

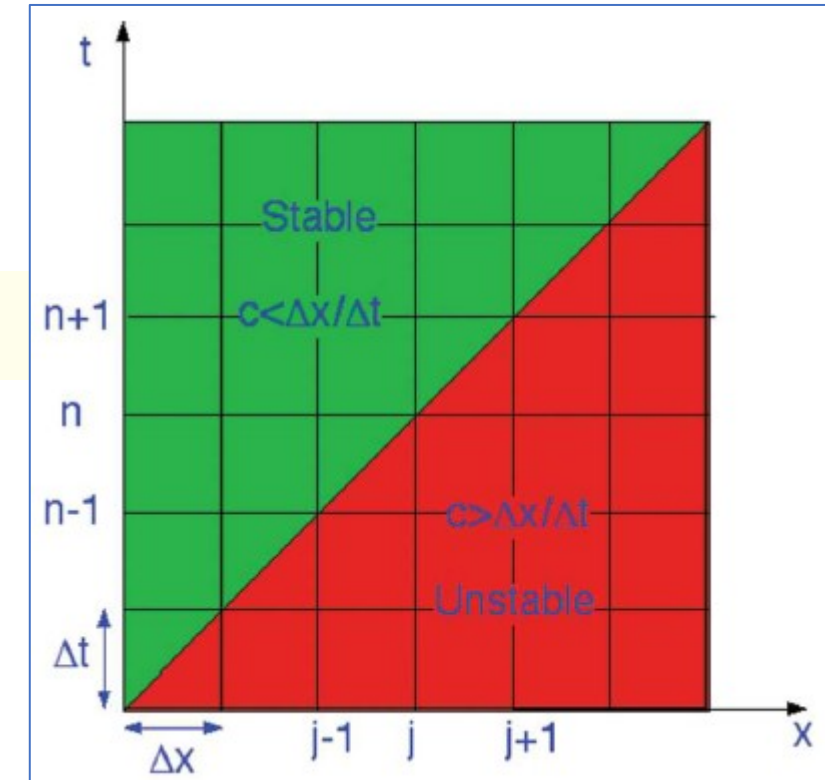
$\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, de modo que $j\Delta x \rightarrow x$ e $n\Delta t \rightarrow t$, portanto, $U(j\Delta x, n\Delta t) \Rightarrow u(x, t)$?

(a) $0 \leq c \leq \frac{\Delta x}{\Delta t}$



$$\mu \equiv \frac{c\Delta t}{\Delta x}$$

$$U_j^{n+1} = (1 - \mu)U_j^n + \mu U_{j-1}^n$$



Esquema da relação entre Δx , Δt e c levando à interpolação da solução no nível de tempo $n + 1$.

Esquema da relação entre Δx , Δt e c levando à interpolação da solução no nível de tempo $n + 1$.



Modelo Numérico da Atmosfera



Convergência e Estabilidade

$\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, de modo que $j\Delta x \rightarrow x$ e $n\Delta t \rightarrow t$, portanto, $U(j\Delta x, n\Delta t) \Rightarrow u(x, t)$?

$$0 < \mu \equiv \frac{c\Delta t}{\Delta x} < 1$$

No entanto, suponha que esta condição não seja satisfeita, de modo que

Duas opções

$$\mu = \frac{c\Delta t}{\Delta x} > 1$$

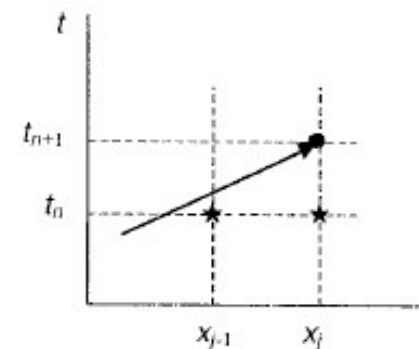
ou

$$\mu = \frac{c\Delta t}{\Delta x} < 0.$$

Então, a parcela que chega ao ponto x_j no tempo t_{n+1} **vem de algum lugar fora do intervalo $(x_j - 1, x_j)$** no tempo t_n .

Lembre-se de que $\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0$ é uma aproximação linear de $\frac{du}{dt} = 0$

(b) $0 \leq \frac{\Delta x}{\Delta t} \leq c$



Então os valores de U_j^{n+1} devem ser **extrapolada** dos valores U_j^n e U_{j-1}^n .



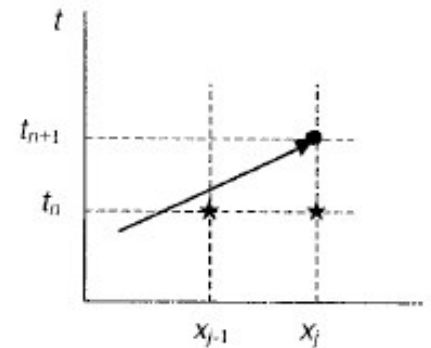
Convergência e Estabilidade

$\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, de modo que $j\Delta x \rightarrow x$ e $n\Delta t \rightarrow t$, portanto, $U(j\Delta x, n\Delta t) \Rightarrow u(x, t)$?

opções

$$\mu = \frac{c\Delta t}{\Delta x} > 1$$

(b) $0 \leq \frac{\Delta x}{\Delta t} \leq c$



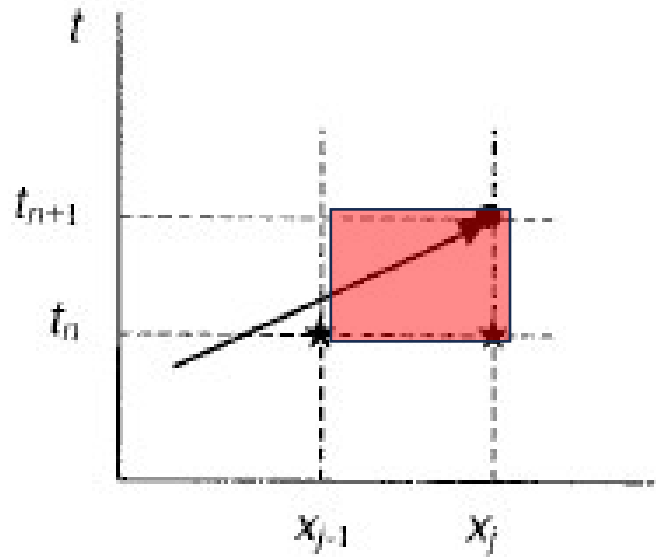
Então os valores de U_j^{n+1} devem ser **extrapolada** dos valores U_j^n e U_{j-1}^n .



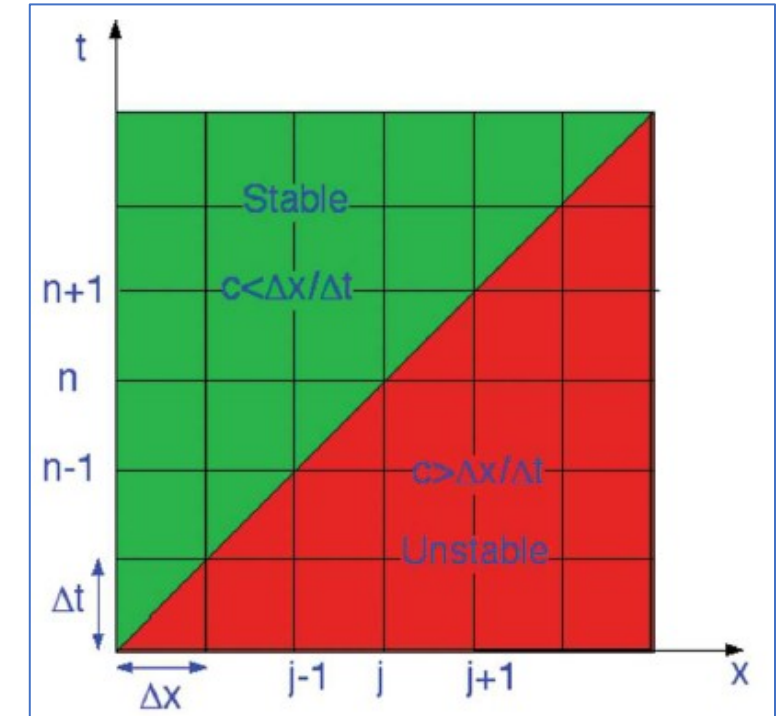
Convergência e Estabilidade

$\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, de modo que $j\Delta x \rightarrow x$ e $n\Delta t \rightarrow t$, portanto, $U(j\Delta x, n\Delta t) \Rightarrow u(x, t)$?

$$(b) 0 \leq \frac{\Delta x}{\Delta t} \leq c$$



$$u_j^{n+1} - u_j^n = -\frac{c\Delta t}{\Delta x} (u_{j-1}^n - u_j^n)$$



Esquema da relação entre Δx , Δt e c liderando à extrapolação da solução no nível de tempo $n + 1$.

$$\mu \equiv \frac{c\Delta t}{|\Delta x|} > 1$$



Convergência e Estabilidade

$\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, de modo que $j\Delta x \rightarrow x$ e $n\Delta t \rightarrow t$, portanto, $U(j\Delta x, n\Delta t) \Rightarrow u(x, t)$?

O problema com a extrapolação é que o máximo valor absoluto da solução U_j^n aumenta a cada intervalo de tempo.

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} = -c \frac{U_j^n - U_{j-1}^n}{\Delta x}$$

$$U_j^{n+1} - U_j^n = -c \frac{\Delta t}{\Delta x} (U_j^n - U_{j-1}^n)$$

$$U_j^{n+1} = U_j^n - c \frac{\Delta t}{\Delta x} U_j^n + c \frac{\Delta t}{\Delta x} U_{j-1}^n$$

$$U_j^{n+1} = \left(1 - c \frac{\Delta t}{\Delta x}\right) U_j^n + c \frac{\Delta t}{\Delta x} U_{j-1}^n$$

$$U_j^{n+1} = (1 - \mu) U_j^n + \mu U_{j-1}^n$$

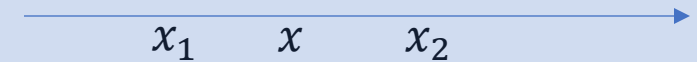
$$\mu = c \frac{\Delta t}{\Delta x}$$

$$\mu = c \frac{\Delta t}{\Delta x}$$

$$(1 - \mu) < 1$$

$$U_j^{n+1} = (1 - \mu) U_j^n + \mu U_{j-1}^n$$

$$f(x) = \left(\frac{x - x_2}{x_2 - x_1}\right) f(x_1) + \left(\frac{x - x_1}{x_2 - x_1}\right) f(x_2)$$



problema com a interpolação

$$U_j^{n+1} = \left(1 - c \frac{\Delta t}{\Delta x}\right) U_j^n + c \frac{\Delta t}{\Delta x} U_{j-1}^n$$

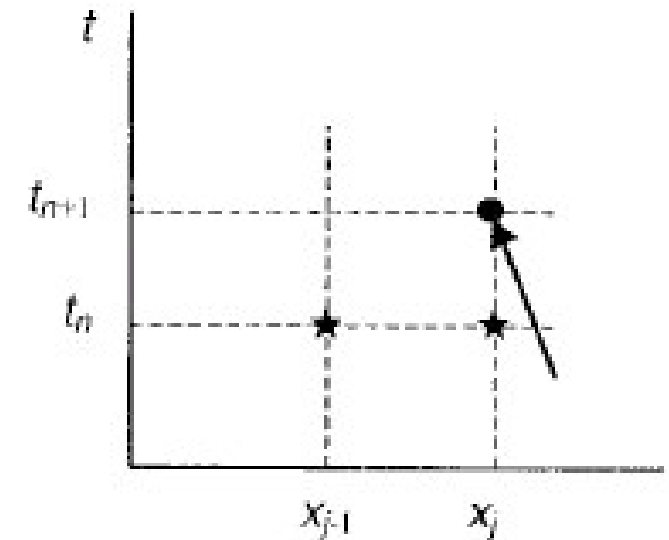


Convergência e Estabilidade

$\Delta x \rightarrow 0$ e $\Delta t \rightarrow 0$, de modo que $j\Delta x \rightarrow x$ e $n\Delta t \rightarrow t$, portanto, $U(j\Delta x, n\Delta t) \Rightarrow u(x, t)$?

$$\mu = \frac{c\Delta t}{\Delta x} < 0.$$

$$(c) \quad c \leq 0 \leq \frac{\Delta x}{\Delta t}$$

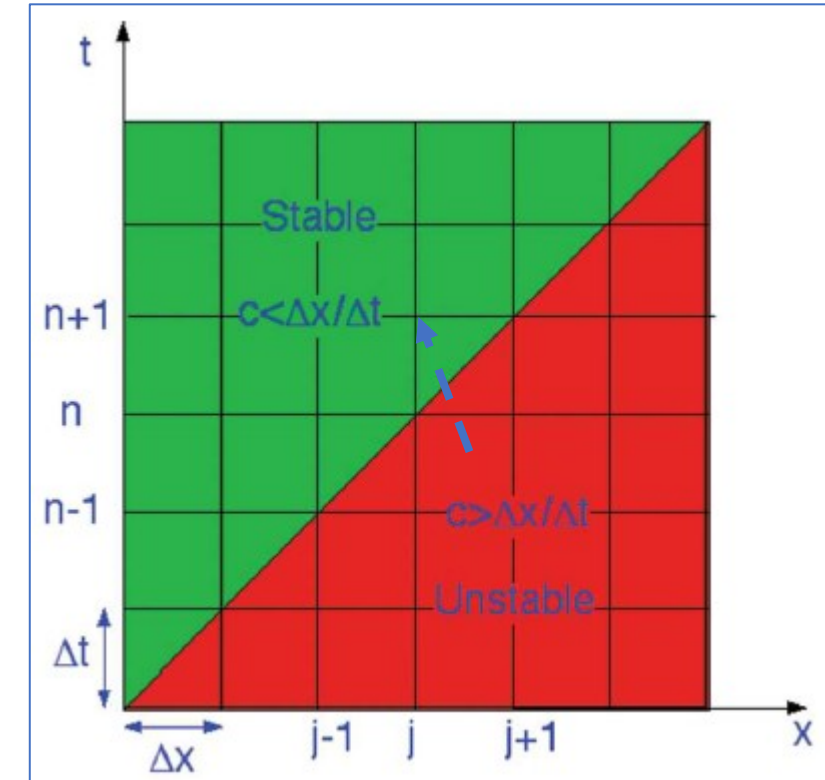
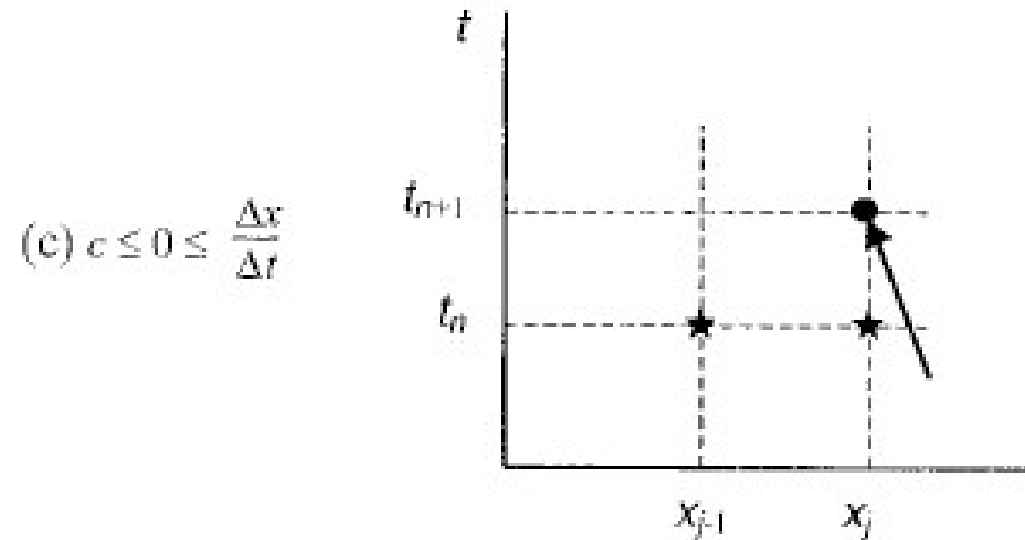


$$U_j^{n+1} = \left(1 - c \frac{\Delta t}{\Delta x}\right) U_j^n + c \frac{\Delta t}{\Delta x} U_{j-1}^n$$

Então os valores de U_j^{n+1} devem ser **extrapolada** dos valores U_j^n e U_{j-1}^n .



Convergência e Estabilidade



Esquema da relação entre Δx , Δt e c liderando à **extrapolação da solução** no nível de tempo $n + 1$.

$$\mu \equiv \frac{c\Delta t}{\Delta x} < 0$$



Convergência e Estabilidade

O problema com a extrapolação é que o máximo valor absoluto da solução U_j^n aumenta a cada intervalo de tempo.

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} = -c \frac{U_j^n - U_{j-1}^n}{\Delta x}$$

$$U_j^{n+1} - U_j^n = -c \frac{\Delta t}{\Delta x} (U_j^n - U_{j-1}^n)$$

$$U_j^{n+1} = U_j^n - c \frac{\Delta t}{\Delta x} U_j^n + c \frac{\Delta t}{\Delta x} U_{j-1}^n$$

$$U_j^{n+1} = \left(1 - c \frac{\Delta t}{\Delta x}\right) U_j^n + c \frac{\Delta t}{\Delta x} U_{j-1}^n$$

$$U_j^{n+1} = (1 - \mu) U_j^n + \mu U_{j-1}^n$$

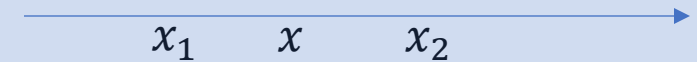
$$\mu = c \frac{\Delta t}{\Delta x}$$

$$\mu \equiv \frac{c \Delta t}{\Delta x} < 0$$

$$(1 - \mu) > 1$$

$$U_j^{n+1} = (1 - \mu) U_j^n + \mu U_{j-1}^n$$

$$f(x) = \left(\frac{x - x_2}{x_2 - x_1}\right) f(x_1) + \left(\frac{x - x_1}{x_2 - x_1}\right) f(x_2)$$



problema com a extrapolação



Convergência e Estabilidade

O problema com a extrapolação é que o máximo valor absoluto da solução U_j^n aumenta a cada intervalo de tempo. Tomando valores absolutos do FDE, obtemos

$$|U_j^{n+1}| \leq |U_j^n| |1 - \mu| + |U_{j-1}^n| |\mu|$$

$$U_j^{n+1} = (1 - \mu)U_j^n + \mu U_{j-1}^n$$

$$\Upsilon^{n+1} = \max_j (U_1^n, U_2^n, U_3^n, \dots, U_j^n)$$

$$\Upsilon^{n+1} = \max_j (U_1^n, U_2^n, U_3^n, \dots, U_{j-1}^n)$$

$$\mu = c \frac{\Delta t}{\Delta x}$$

Agora definindo $\Upsilon^{n+1} = \max_j |U_j^n|$; $\Upsilon^{n+1} = \max_j |U_{j-1}^n|$; $\Upsilon^{n+2} = \max_j |U_j^{n+1}|$ tem-se:

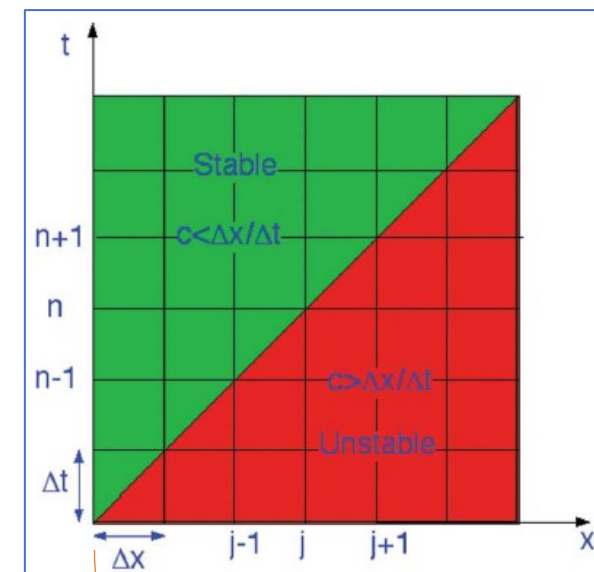
$$\max_j |U_j^{n+1}| \leq \max_j |U_j^n| |1 - \mu| + \max_j |U_{j-1}^n| |\mu|$$

$$\Upsilon^{n+2} \leq \Upsilon^{n+1} |1 - \mu| + \Upsilon^{n+1} |\mu|$$

$$\Upsilon^{n+1} \Upsilon^1 \leq \Upsilon^n \Upsilon^1 |1 - \mu| + \Upsilon^n \Upsilon^1 |\mu|$$

$$\Upsilon^{n+1} \leq \Upsilon^n |1 - \mu| + \Upsilon^n |\mu|$$

$$\Upsilon^{n+1} \leq \{|1 - \mu| + |\mu|\} \Upsilon^n$$





Convergência e Estabilidade

- problema com a extrapolação é que o máximo valor absoluto da solução U_j^n aumenta a cada intervalo de tempo.

Tomando valores absolutos do FDE, obtemos



Convergência e Estabilidade

$$U_j^{n+1} = (1 - \mu)U_j^n + \mu U_{j-1}^n$$

$$\Upsilon^{n+1} \leq \{|1 - \mu| + |\mu|\}\Upsilon^n$$

$$\mu = c \frac{\Delta t}{\Delta x}$$

e $\Upsilon^{n+1} \leq \Upsilon^n$ se e somente se $0 \leq \mu \leq 1$.

Se a condição $0 \leq \mu \leq 1$ não for satisfeita, a solução não é limitada e cresce com n .

Se deixarmos $\Delta x \rightarrow 0, \Delta t \rightarrow 0$ com $\mu = \text{const.}$, Isso só piorará as coisas, porque então $n \rightarrow \infty$.

Na prática, se a condição $0 \leq \mu \leq 1$ não for satisfeita, o FDE explode em alguns intervalos de tempo.



Convergência e Estabilidade

Estabilidade Computacional



Convergência e Estabilidade

Exercício prático:

Use o **modelo simples SLAM** para explorar o fenômeno da instabilidade computacional.

$$U_j^{n+1} = (1 - \mu)U_j^n + \mu U_{j-1}^n$$

- Escolha Δt pequeno e faça uma integração completa.
- Aumente Δt e observe a solução do modelo “explodir”.
- Para experimento numérico, **determine aproximadamente o valor máximo de Δt que produz integrações estáveis.**
- Relacione este máximo com o **número Courant**. O que isso implica sobre a velocidade máxima de fase do sistema?

$$\mu = c \frac{\Delta t}{\Delta x}$$



Estabilidade Computacional

Agora definimos estabilidade computacional.

Definição: Um FDE é **computacionalmente estável** se a solução do FDE em um tempo fixo $t = n\Delta t$ é limitado como $\Delta t \rightarrow 0$.

Observe que com $n\Delta t$ fixo, $\Delta t \rightarrow 0$ implica $n \rightarrow \infty$

$$\mu = c \frac{\Delta t}{\Delta x}$$

Iremos derivar uma condição de estabilidade que envolve o **Courant Number**.

A condição no número de Courant é geralmente conhecida como critério **Courant-Friedrichs-Lewy** ou simplesmente a condição **CFL**.

###

Lembre-se da história de Courant, Friedrichs e Lewy em Göttingen.



Estabilidade Computacional

Agora podemos afirmar o teorema fundamental de Lax-Richtmyer:

Dado um problema de valor inicial linear corretamente proposto e um esquema de diferenças finitas que satisfaça a condição de consistência, então a estabilidade do FDE é a condição necessária e suficiente para a convergência.

$$\text{Estabilidade} < = > \text{Convergencia}$$

Para sistemas consistentes

Este teorema nos permite estabelecer **convergência** examinando as questões mais fáceis de **consistência** e **estabilidade**.

Estamos interessados na convergência não porque queremos deixar $\Delta x, \Delta t \rightarrow 0$, mas porque queremos ter certeza de que os erros $u(j\Delta x, n\Delta t) - U_j^n$ sejam aceitavelmente pequenos.

Definição $u(j\Delta x, n\Delta t) - U_j^n$ é o **erro de truncamento global**.



Estabilidade Computacional

Exemplo: Usamos o critério do método máximo para estudar a condição de estabilidade da equação de difusão

$$\frac{\partial u}{\partial t} = \sigma \frac{\partial^2 u}{\partial x^2}$$

Uma aproximação FDE (esquema FTCS) é dada por

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} = \sigma \frac{U_{j+1}^n - 2U_j^n + U_{j-1}^n}{\Delta x^2}$$

(a verificação da consistência FDE é imediata).

Nota: Uma vez que as diferenças são centradas no espaço, mas avançadas no tempo, o erro de truncamento é de primeira ordem no tempo e de segunda ordem no espaço

$$\tau = O(\Delta t) + O(\Delta x)^2.$$

Podemos escrever o FDE na forma de equação

$$U_j^{n+1} = \mu U_{j+1}^n + (1 - 2\mu)U_j^n + \mu U_{j-1}^n$$

$$\mu = \sigma \Delta t / \Delta x^2.$$



Estabilidade Computacional

Novamente

$$U_j^{n+1} = \mu U_{j+1}^n + (1 - 2\mu)U_j^n + \mu U_{j-1}^n$$

Se tomarmos valores absolutos, e deixarmos $\Upsilon^n = \max_j |U_j^n|$

$$\Upsilon^{n+1} \leq \{|\mu| + |1 - 2\mu| + |\mu|\}\Upsilon^n$$

$$\{|1 - 2\mu|\} \leq 2|\mu|$$

Assim, obtemos a condição

$$\left\{\left|\frac{1}{2} - \mu\right|\right\} \leq |\mu|$$

$$0 \leq \mu \leq 1/2$$

$$0 \leq \mu \leq 1/2$$

para garantir que a solução permaneça limitada como $n \rightarrow \infty$.

Essa é a condição necessária para a estabilidade do FDE.

###

Infelizmente, o **critério do máximo** só pode ser aplicado em poucos casos.

Na maioria dos FDEs, alguns coeficientes das equações são negativos e o critério não pode ser aplicado.

Precisamos de **um método mais poderoso** de estabelecer estabilidade.



Modelo Numérico da Atmosfera



Método de von Neumann — Análise de Estabilidade

Ideia central: substituir um modo de Fourier $U_j^n = Z^n \cdot e^{ipj}$ no FDE e calcular o fator de amplificação $G = Z^{n+1}/Z^n$. Se $|G| \leq 1$ para todo $p \rightarrow$ estável. Se $|G| > 1 \rightarrow$ instável.

1

Substituir o modo de Fourier

Coloque $U_j^n = Z^n \cdot e^{ipj}$ na FDE.
Cancela e^{ipj} de ambos os lados.

2

Calcular G

Isole $G = Z^{n+1}/Z^n$.
G depende de p e dos parâmetros ($\mu, r, \dots$).

3

Verificar $|G| \leq 1$

Para todo $p \in [-\pi, \pi]$.
Encontre a condição sobre Δt e Δx .

Exemplo 1 — Esquema Upstream (advecção)

Substituição:

$$Z^{n+1}e^{ipj} = (1-\mu)Z^ne^{ipj} + \mu Z^ne^{ip(j-1)} \rightarrow \\ G = 1 - \mu + \mu e^{-ip}$$

Módulo $|G|^2$:

$$|G|^2 = 1 - 2\mu(1-\mu)(1-\cos p) \leq 1 \quad \text{se} \quad 0 \leq \mu \leq 1$$

Condição CFL: $\mu = c\Delta t/\Delta x \leq 1 \rightarrow$ coincide com a análise geométrica!

Exemplo 2 — Equação do Calor (difusão FTCS)

Esquema:

$$U_j^{n+1} = U_j^n + r(U_{j+1}^n - 2U_j^n + U_{j-1}^n) \\ r = \alpha\Delta t/\Delta x^2$$

Fator G:

$$G = 1 - 4r \cdot \sin^2(p/2) \rightarrow |G| \leq 1 \quad \text{se} \quad r \leq 1/2$$

Condição: $r = \alpha\Delta t/\Delta x^2 \leq 1/2 \rightarrow \Delta t \leq \Delta x^2/(2\alpha)$



O Método de von Neumann

Outro critério de estabilidade com uma aplicação muito mais ampla é o **critério de estabilidade de von Neumann**. Assumimos que podemos expandir a solução do FDE em um conjunto apropriado de autofunções.

Para simplificar, assumimos **uma expansão para a série Fourier**:

$$U(x, t) = \sum_k Z_k(t) e^{ikx}$$

$$U_j^n = \sum_k Z_k^n e^{ikj\Delta x}$$

A variável de espaço x e o número de onda k podem ser multidimensionais, $x = (x_1, x_2, x_3)$ e $k = (k_1, k_2, k_3)$

mas, para simplificar, consideraremos o caso escalar. Nós definimos $x_j = j\Delta x$, $t_n = n\Delta t$

Definimos o número de onda para a série Fourier: $p = k\Delta x$

Então a expansão de Fourier é

$$U_j^n = \sum_p Z_p^n e^{ipj} \quad (\text{Note: } kx = kj\Delta x = pj)$$



O Método de von Neumann

$$U_j^n = \sum_p Z_p^n e^{ipj} \quad (\text{Note: } kx = kj\Delta x = pj)$$

Quando substituimos esta expansão de Fourier em um FDE linear, obtemos um sistema de equações

$$Z_p^{n+1} = \rho_p Z_p^n \quad \rho_p = e^{ipj}$$

Aqui ρ_p é um fator de amplificação que, aplicado ao p-ésimo componente de Fourier da solução no tempo $n\Delta t$, o avança para o tempo $(n+1)\Delta t$; ρ_p depende de p , Δt e Δx .

Se conhecermos as **condições iniciais**

$$U_j^0 = \sum_p Z_p^0 e^{ipj}$$

então a solução do FDE é (lembre-se do aviso sobre sobrescritos)

$$Z_p^n = \rho_p^n Z_p^0$$

Portanto, a estabilidade é garantida se o fator **ρ^n for limitado para todo** $p = k\Delta x$ quando $\Delta t \rightarrow 0$ e $n \rightarrow \infty$

Portanto, devemos ter $|\rho_p| < \mu$ para todo $p = k\Delta x$ como $n \rightarrow \infty$.



Métodos de Diferenças Finitas

Consistência e Estabilidade da solução numérica

Teorema de Lax (equivalência):

Para problemas lineares bem-postos, **Consistência + Estabilidade $\Leftrightarrow$ Convergência**

Consistência

Condição necessária

O erro de truncamento $\tau \rightarrow 0$ quando $\Delta t \rightarrow 0$ e $\Delta x \rightarrow 0$.

Garante que o FDE representa corretamente o PDE.

Sozinha NÃO garante convergência da solução.

Estabilidade

Condição necessária e suficiente

O erro não cresce sem controle durante a integração temporal.

Erros de round-off e truncamento acumulam a cada passo Δt .

Δt pequeno $\rightarrow$ acúmulo mais lento, mas não elimina.

Convergência

Objetivo final

A solução numérica $U(j\Delta x, n\Delta t) \rightarrow$ solução exata $u(x,t)$ quando $\Delta t, \Delta x \rightarrow 0$.

Garantida pelo Teorema de Lax se consistente E estável.

É o que buscamos em NWP.

Atenção: se o método é instável, a solução numérica diverge rapidamente da solução correta — independente do tamanho de Δt .



Métodos de diferenças finitas.

Análise de Estabilidade EDO linear

Assume-se que a solução para a eq. 7 tem a forma $y^n = A^n e^{ik}$ e substitua no Esquema Avançado Eq. 9

$$\frac{\partial y}{\partial t} = -\lambda y \quad (7)$$

$$\frac{y^{n+1} - y^n}{\Delta t} = -\lambda y^n \quad (7)$$

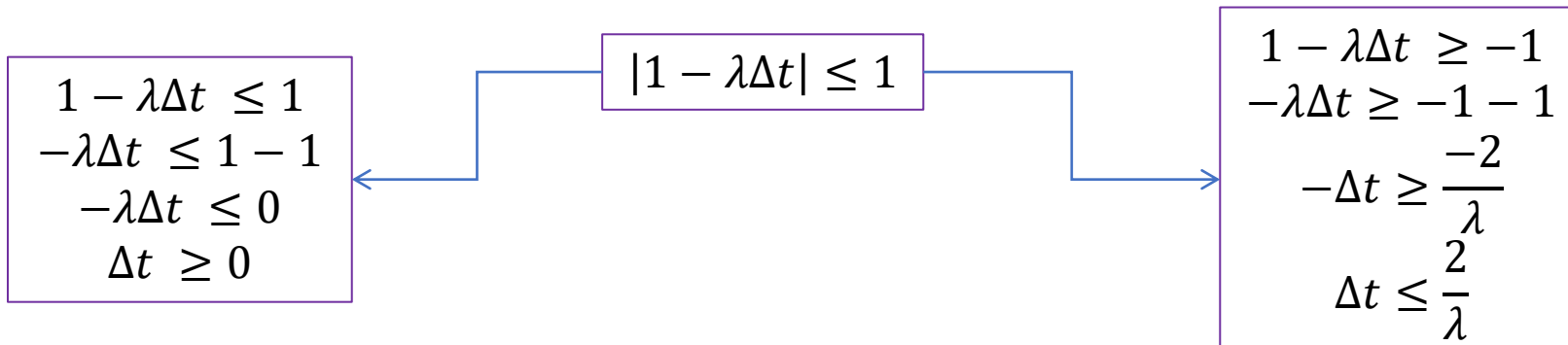
$$y^{n+1} = (1 - \lambda \Delta t) y^n \quad (9)$$

$$A^{n+1} e^{ik} = (1 - \lambda \Delta t) A^n e^{ik} \quad (10)$$

$$A^n A e^{ik} = A^n e^{ik} (1 - \lambda \Delta t) \quad (11)$$

$$A = (1 - \lambda \Delta t) \quad (12)$$

A é chamado de fator de amplificação. Então $|A| \leq 1$





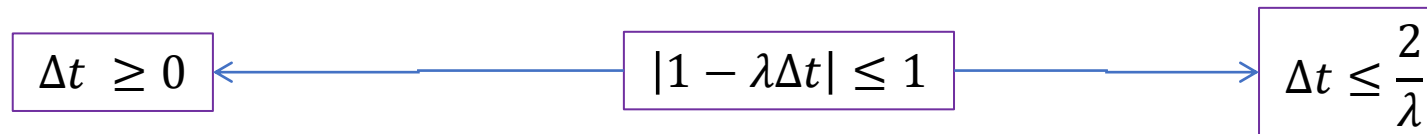
Métodos de diferenças finitas.

Análise de Estabilidade EDO linear

Assume-se que a solução para a eq. 7 tem a forma $y^n \approx A^n$ e substitua no Esquema Avançado Eq. 9

$$A = (1 - \lambda \Delta t) \quad (12)$$

O sistema é estável, se



N.B. Para uma equação não linear, a análise de estabilidade da forma linearizada da equação não linear é uma condição necessária, mas não é suficiente.



Métodos de Diferenças Finitas



Implementação em Fortran — EDO Linear: $dy/dt = -\lambda y$ (esquema forward)

Objetivo: implementar o esquema Forward Euler $y^{n+1} = y^n - \lambda \Delta t \cdot y^n = (1 - \lambda \Delta t) y^n$ e comparar com a solução exata $y(t) = y_0 \cdot e^{(-\lambda t)}$

MODULE — estrutura e variáveis

```
MODULE Class_NumericalScheme
  IMPLICIT NONE
  PRIVATE
  INTEGER, PUBLIC, PARAMETER :: r8=8
  INTEGER, PUBLIC, PARAMETER :: r4=4
  REAL (KIND=r8) :: dt
  REAL (KIND=r8) :: lambda
  INTEGER, PARAMETER :: UnitData=1
  INTEGER, PARAMETER :: UnitCtl=2
  LOGICAL :: CtrlWriteDataFile
  PUBLIC :: InitNumericalScheme
  PUBLIC :: SchemeForward
  PUBLIC :: SchemeUpdate
  PUBLIC :: AnaliticFunction
CONTAINS
SUBROUTINE InitNumericalScheme(dt_in,lambda_in)
  IMPLICIT NONE
  REAL (KIND=r8) :: dt_in
  REAL (KIND=r8) :: lambda_in
  dt=dt_in
  lambda=lambda_in
  CtrlWriteDataFile=.TRUE.
END SUBROUTINE InitNumericalScheme
```

Encapsula dt , λ e métodos.
PARAMETER $r8=8 \rightarrow$ dupla precisão.
CONTAINS separa interface da implementação.

SchemeForward — esquema numérico

```
FUNCTION SchemeForward(yc) RESULT(y_p)
  IMPLICIT NONE
  ! Diferenciacao forward:
  ! y(n+1) = y(n) - lambda*dt*y(n)
  REAL(KIND=r8), INTENT(IN) :: yc
  REAL(KIND=r8) :: y_p
  y_p = yc - lambda*dt*yc
END FUNCTION SchemeForward

FUNCTION SchemeUpdate(y_in) RESULT(y_out)
  IMPLICIT NONE
  REAL(KIND=r8), INTENT(IN) :: y_in
  REAL(KIND=r8) :: y_out
  y_out = y_in
END FUNCTION SchemeUpdate
```

$y_p = y_c - \lambda \cdot dt \cdot y_c$
 $= (1 - \lambda \Delta t) \cdot y_c$
Fator de amplificação: $G = 1 - \lambda \Delta t$
Estável se $|G| \leq 1 \rightarrow \Delta t \leq 2/\lambda$

AnaliticFunction — solução exata

```
FUNCTION AnaliticFunction(y0,tn) RESULT(y_out)
  IMPLICIT NONE
  REAL(KIND=r8), INTENT(IN) :: y0
  INTEGER, INTENT(IN) :: tn
  REAL(KIND=r8) :: y_out
  ! Solucao exata: y(t) = y0 * exp(-lambda*t)
  y_out = y0*exp(-lambda*tn*dt)
END FUNCTION AnaliticFunction
```

$y(t) = y_0 \cdot \exp(-\lambda t)$
Usada para validar o esquema numérico comparando com y_p .

Erro numérico

$erro(n) = |y^n - y(n\Delta t)|$

Cresce se $\Delta t > 2/\lambda$
(instabilidade)



Métodos de diferenças finitas.

```
MODULE Class_NumericalScheme
  IMPLICIT NONE
  PRIVATE
  INTEGER, PUBLIC , PARAMETER :: r8=8
  INTEGER, PUBLIC , PARAMETER :: r4=4

  REAL (KIND=r8) :: dt
  REAL (KIND=r8) :: lambda
  INTEGER , PARAMETER :: UnitData=1
  INTEGER , PARAMETER :: UnitCtl=2
  CHARACTER (LEN=400) :: FileName
  LOGICAL :: CtrlWriteDataFile
  PUBLIC :: InitNumericalScheme
  PUBLIC :: SchemeForward
  PUBLIC :: SchemeUpdate
  PUBLIC :: SchemeWriteData
  PUBLIC :: SchemeWriteCtl
  PUBLIC :: AnaliticFunction

CONTAINS

  SUBROUTINE InitNumericalScheme(dt_in,lambda_in)
    IMPLICIT NONE
    REAL (KIND=r8) :: dt_in
    REAL (KIND=r8) :: lambda_in
    FileName=""
    dt=dt_in
    lambda=lambda_in
    FileName='EDOL'
    CtrlWriteDataFile=.TRUE.
  END SUBROUTINE InitNumericalScheme
```

```
FUNCTION SchemeForward(y) RESULT(y)
  IMPLICIT NONE
  ! Utilizando a diferenciacao forward
  !
  !  $y(n+1) - y(n)$ 
  ! ----- = lambda * y(n) ; onde lambda > 0
  !      dt
  !
  REAL (KIND=r8), INTENT (IN ) :: y
  REAL (KIND=r8) :: yp

  yp = y -lambda*dt*y

END FUNCTION SchemeForward

FUNCTION SchemeUpdate(y_in) RESULT (y_out)
  IMPLICIT NONE
  REAL (KIND=r8), INTENT(IN) :: y_in
  REAL (KIND=r8) :: y_out
  y_out=y_in
END FUNCTION SchemeUpdate

FUNCTION AnaliticFunction(y0,tn) RESULT (y_out)
  IMPLICIT NONE
  REAL (KIND=r8), INTENT (IN) :: y0
  INTEGER , INTENT (IN) :: tn

  REAL (KIND=r8) :: y_out
  y_out=y0*exp(-lambda*tn*dt)
END FUNCTION AnaliticFunction
```



Métodos de diferenças finitas.

```
FUNCTION SchemeWriteData(irec,y_in,ya) RESULT (ok)
```

```
IMPLICIT NONE
```

```
INTEGER , INTENT (INOUT) :: irec
```

```
REAL (KIND=r8), INTENT (IN) :: y_in
```

```
REAL (KIND=r8), INTENT (IN) :: ya
```

```
INTEGER :: ok
```

```
INTEGER :: lrec
```

```
REAL (KIND=r4) :: Yout
```

```
INQUIRE (IOLENGTH=lrec) Yout
```

```
IF(CtrlWriteDataFile)OPEN(UnitData,FILE=TRIM(FileName)//'.bin',&
```

```
FORM='UNFORMATTED', ACCESS='DIRECT',
```

```
STATUS='UNKNOWN', &
```

```
ACTION='WRITE',RECL=lrec)
```

```
CtrlWriteDataFile=.FALSE.
```

```
Yout=REAL(y_in,KIND=r4)
```

```
irec=irec+1
```

```
WRITE(UnitData,rec=irec) Yout
```

```
irec=irec+1
```

```
WRITE(UnitData,rec=irec) REAL (ya,KIND=r4)
```

```
ok=0
```

```
END FUNCTION SchemeWriteData
```

```
FUNCTION SchemeWriteCtl(nrec) RESULT( ok)
```

```
IMPLICIT NONE
```

```
INTEGER, INTENT (IN) :: nrec
```

```
INTEGER :: ok
```

```
OPEN(UnitCtl,FILE=TRIM(FileName)//'.ctl',FORM='FORMATTED',&  
ACCESS='SEQUENTIAL',STATUS='UNKNOWN',ACTION='WRITE')
```

```
WRITE (UnitCtl,'(A6,A)')'dset ^',TRIM(FileName)//'.bin'
```

```
WRITE (UnitCtl,'(A )')'title EDO'
```

```
WRITE (UnitCtl,'(A )')'undef -9999.9'
```

```
WRITE (UnitCtl,'(A )')'xdef 1 linear -48.00 1'
```

```
WRITE (UnitCtl,'(A )')'ydef 1 linear -1.27 1'
```

```
WRITE (UnitCtl,'(A6,I6,A25)')'tdef ',nrec,' linear 00z01jan0001 1hr'
```

```
WRITE (UnitCtl,'(A20 )')'zdef 1 levels 1000 '
```

```
WRITE (UnitCtl,'(A)')'vars 2'
```

```
WRITE (UnitCtl,'(A)')'yc 0 99 resultado da edol yc'
```

```
WRITE (UnitCtl,'(A)')'ya 0 99 funcao analitica'
```

```
WRITE (UnitCtl,'(A)')'endvars'
```

```
CLOSE (UnitCtl,STATUS='KEEP')
```

```
CLOSE (UnitData,STATUS='KEEP')
```

```
ok=0
```

```
END FUNCTION SchemeWriteCtl
```

```
END MODULE Class_NumericalScheme
```




Métodos de diferenças finitas.



PROGRAM MAIN

```
USE Class_NumericalScheme, Only :r8, InitNumericalScheme, SchemeForward, SchemeUpdate,&  
    SchemeWriteCtl, SchemeWriteData, AnaliticFunction
```

IMPLICIT NONE

```
REAL (KIND=r8) :: yn  
REAL (KIND=r8) :: yc  
REAL (KIND=r8) :: yp  
REAL (KIND=r8) :: ya
```

```
REAL (KIND=r8), PARAMETER :: lambda=0.1 ![1/s]  
REAL (KIND=r8), PARAMETER :: y0=1.0_r8  
INTEGER , PARAMETER :: nrec=200  
REAL(KIND=r8) :: dt=1.5/lambda  
INTEGER :: test  
INTEGER :: irec  
INTEGER :: tn
```

CALL Init()

```
yc=y0
```

```
irec=0
```

DO tn=0,nrec

```
    yp=SchemeForward(yc)  
    ya=AnaliticFunction(y0,tn)  
    test=SchemeWriteData(irec,yc,ya)  
    yn=SchemeUpdate(yc)  
    yc=SchemeUpdate(yp)
```

END DO

```
test=SchemeWriteCtl(nrec)
```

CONTAINS

SUBROUTINE Init()

IMPLICIT NONE

```
CALL InitNumericalScheme(dt,lambda)
```

END SUBROUTINE Init

END PROGRAM MAIN

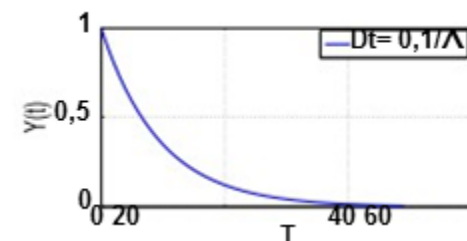


Métodos de diferenças finitas.

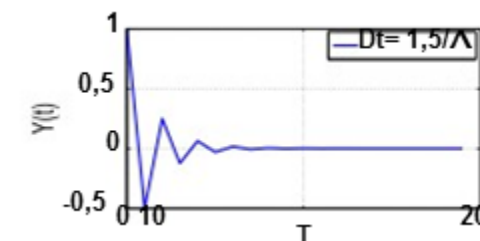


Há três possibilidades para a solução numérica ou seja $y^{n+1} = (1 - \lambda\Delta t)y^n$ depende a escolha de Δt

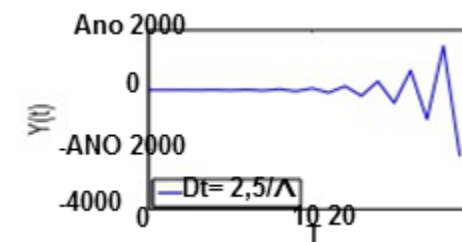
I) Se $\Delta t < \frac{1}{\lambda}$, então $0 < (1 - \lambda\Delta t) < 1$, A solução numérica é uma função decrescente com o tempo



II) Se $\frac{1}{\lambda} < \Delta t < \frac{2}{\lambda}$, A solução numérica diminui a magnitude mas oscila no sinal. O sistema é estável, mas não consistente.



(III) se $\Delta t > \frac{2}{\lambda}$ então, $(1 - \lambda\Delta t) < -1$, A solução oscila no sinal e aumenta em magnitude com o passar do tempo. O método é instável para as grandes passos de tempo.





Métodos de diferenças finitas.

A restrição de Δt para garantir a estabilidade pode por vezes ser removido pela escolha do método numérico. ex. usando o esquema **atrasado**.

$$\frac{dy}{dt} = -\lambda y$$

$$\frac{y^n - y^{n-1}}{\Delta t} = -\lambda y^n$$

$$A^n - A^{n-1} = -\lambda \Delta t A^n$$

$$A^n - A^n A^{-1} = -\lambda \Delta t A^n$$

$$\rightarrow 1 - A^{-1} = -\lambda \Delta t$$

$$A^{-1} = 1 + \lambda \Delta t$$

$$A = \frac{1}{1 + \lambda \Delta t}$$

O sistema é estável, se $|A| \leq 1$

$$|A| = \left| \frac{1}{1 + \lambda \Delta t} \right| \leq 1$$



Métodos de diferenças finitas.

Análise de Estabilidade EDO linear

Assume-se que a solução para a eq. 7 tem a forma $y^n = A^n e^{ik}$ e substitua no Esquema Avançado Eq. 9

$$\frac{\partial y}{\partial t} = -\lambda y \quad (7)$$

$$\frac{y^{n+1} - y^{n-1}}{\Delta t} = -\lambda y^n \quad (7)$$

$$y^{n+1} = (1y^{n-1} - \lambda \Delta t y^n) \quad (9)$$

$$A^{n+1} e^{ik} = (A^{n-1} - \lambda \Delta t A^n) e^{ik} \quad (10)$$

$$A^n A^1 e^{ik} = (A^n A^{-1} - \lambda \Delta t A^n) e^{ik} \quad (11)$$

$$A^n A^1 e^{ik} = A^n (A^{-1} - \lambda \Delta t) e^{ik} \quad (12)$$

$$A^1 = (A^{-1} - \lambda \Delta t) \quad (13)$$

$$A - \frac{1}{A} = (-\lambda \Delta t) \quad (14)$$

$$\frac{1}{A} (A^2 - 1) = (-\lambda \Delta t) \quad (15)$$

$$(A^2 - 1) = (-\lambda \Delta t) A \quad (16)$$

$$(A^2 + (\lambda \Delta t) A - 1) = 0 \quad (17)$$

A é chamado de fator de amplificação. Então $|A| \leq 1$

$$ax^2 + bx + c = 0$$

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$$\left(A = \frac{-1 \pm \sqrt{(\lambda \Delta t)^2 - 4(1)(-1)}}{2} \right)$$

$$\left(A = \frac{-1}{2} \pm \frac{\sqrt{(\lambda \Delta t)^2 + 4}}{2} \right)$$

$$\left(A = \frac{-1}{2} \pm \sqrt{(\lambda \Delta t)^2 \frac{1}{2^2} + 1} \right)$$



Métodos de diferenças finitas.



Análise de Estabilidade EDO linear

$$\Delta t \leq \frac{2}{\lambda}$$

Assume-se que a solução para a eq. 7 tem a forma $y^n = A^n e^{ik}$ e substitua no Esquema Avançado Eq. 9

$$\left(A = \frac{-1}{2} + \sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \right)$$

$$\left(A = \frac{-1}{2} - \sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \right)$$

A é chamado de fator de amplificação. Então $|A| \leq 1$

$$|A| = \left| \frac{-1}{2} + \sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \right| \leq 1$$

$$\sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \leq \frac{3}{2}$$

$$\frac{\lambda \Delta t}{2} \leq \sqrt{\frac{5}{4}}$$

$$\frac{-1}{2} + \sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \leq 1$$

$$\left(\frac{\lambda \Delta t}{2}\right)^2 + 1 \leq \frac{9}{4}$$

$$\Delta t \leq \frac{2}{\lambda} \sqrt{\frac{5}{4}}$$

$$\left(\frac{\lambda \Delta t}{2}\right)^2 \leq \frac{9}{4} - 1$$

$$\Delta t \leq \frac{1}{\lambda} \sqrt{\frac{4 * 5}{4}}$$

$$\sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \leq 1 + \frac{1}{2}$$

$$\left(\frac{\lambda \Delta t}{2}\right)^2 \leq \frac{5}{4}$$

$$\Delta t \leq \frac{\sqrt{5}}{\lambda}$$



Análise de Estabilidade EDO linear

$$\Delta t \leq \frac{2}{\lambda}$$

Assume-se que a solução para a eq. 7 tem a forma $y^n = A^n e^{ik}$ e substitua no Esquema Avançado Eq. 9

$$\left(A = \frac{-1}{2} + \sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \right)$$

A é chamado de fator de amplificação. Então $|A| \leq 1$

$$\left(A = \frac{-1}{2} - \sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \right)$$

$$|A| = \left| \frac{-1}{2} - \sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \right| \leq 1$$

$$\sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \leq -\frac{3}{2}$$

$$\frac{\lambda \Delta t}{2} \leq \sqrt{\frac{5}{4}}$$

$$\frac{-1}{2} - \sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \leq 1$$

$$\left(\frac{\lambda \Delta t}{2}\right)^2 + 1 \leq \frac{9}{4}$$

$$\Delta t \leq \frac{2}{\lambda} \sqrt{\frac{5}{4}}$$

$$\left(\frac{\lambda \Delta t}{2}\right)^2 \leq \frac{9}{4} - 1$$

$$\Delta t \leq \frac{1}{\lambda} \sqrt{\frac{4 * 5}{4}}$$

$$-\sqrt{\left(\frac{\lambda \Delta t}{2}\right)^2 + 1} \leq 1 + \frac{1}{2}$$

$$\left(\frac{\lambda \Delta t}{2}\right)^2 \leq \frac{5}{4}$$

$$\Delta t \leq \frac{\sqrt{5}}{\lambda}$$



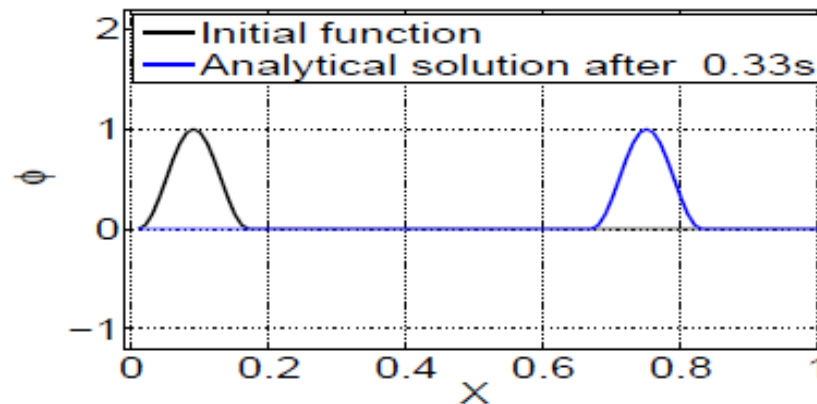
Métodos de diferenças finitas.

Exemplo: Equações diferenciais Parciais

Considere uma equação de **Advecção Linear** em uma dimensão

$$\frac{\partial \phi}{\partial t} + u \frac{\partial \phi}{\partial x} = 0 \quad (10)$$

Onde **u** é a velocidade constante, o domínio é $0 \leq x \leq 1$, com condição de fronteira periódicas $\phi(x, 0) = \phi(1, t)$, E a condição inicial é dada como $\phi(x, 0) = F(x)$. Por exemplo $F(x) = \sin^2(x)$



A solução analítica é

$$\phi(x, t) = F(x - u * t).$$

A função inicial é advectada com velocidade u , e a forma é preservada.



Métodos de diferenças finitas.



```
MODULE Class_Fields
  IMPLICIT NONE
  PRIVATE
  INTEGER, PUBLIC , PARAMETER :: r8=8
  INTEGER, PUBLIC , PARAMETER :: r4=4
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: PHI_P(:)
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: PHI_A(:)
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: PHI_C(:)
  REAL (KIND=r8),PUBLIC ,ALLOCATABLE :: PHI_M(:)
  REAL (KIND=r8),PUBLIC :: Uvel
  INTEGER ,PUBLIC :: iMax
  PUBLIC :: Init_Class_Fields
```

CONTAINS

```
!-----
SUBROUTINE Init_Class_Fields(xdim,Uvel0)
  IMPLICIT NONE
  INTEGER , INTENT (IN ) :: xdim
  REAL (KIND=r8), INTENT (IN ):: Uvel0
  iMax=xdim
  Uvel=Uvel0
  ALLOCATE (PHI_A(-1:iMax+2))
  ALLOCATE (PHI_P(-1:iMax+2))
  ALLOCATE (PHI_C(-1:iMax+2))
  ALLOCATE (PHI_M(-1:iMax+2))
END SUBROUTINE Init_Class_Fields
!-----
END MODULE Class_Fields
```

```
MODULE Class_NumericalMethod
  USE Class_Fields, Only : PHI_A,PHI_P,PHI_C,PHI_M,Uvel,iMax
  IMPLICIT NONE
  PRIVATE
  INTEGER, PUBLIC , PARAMETER :: r8=8
  INTEGER, PUBLIC , PARAMETER :: r4=4
  REAL (KIND=r8) :: Dt
  REAL (KIND=r8) :: Dx
```

```
PUBLIC :: InitNumericalScheme
PUBLIC :: SchemeForward
PUBLIC :: SchemeUpdate
PUBLIC :: SchemeUpStream
PUBLIC :: AnalyticFunction
```

CONTAINS

```
!-----
SUBROUTINE InitNumericalScheme(dt_in,dx_in)
  IMPLICIT NONE
  REAL (KIND=r8), INTENT (IN ) :: dt_in
  REAL (KIND=r8), INTENT (IN ) :: dx_in
  INTEGER :: i
  REAL (KIND=r8) :: x
  Dt=dt_in
  Dx=dx_in
  x=0.0
  DO i=1,iMax
    PHI_C(i)= sin(x)*sin(x)
    x = (6*i)* DX
  END DO
  PHI_M=PHI_C
  PHI_P=PHI_C
END SUBROUTINE InitNumericalScheme
```




Métodos de diferenças finitas.

```

FUNCTION AnaliticFunction(tn) RESULT (ok)
  IMPLICIT NONE
  INTEGER, INTENT (IN) :: tn
  INTEGER :: j,ok
  REAL (KIND=r8) :: x
  ok=1
  x=0.0
  DO j=1,iMax
    PHI_A(j)=(sin(x - Uvel*tn*(dt))**2)
    x = (6*j)* DX
  END DO
  ok=0
END FUNCTION AnaliticFunction

```

```

FUNCTION SchemeForward() RESULT(ok)
  IMPLICIT NONE
  ! Utilizando a diferenciacao forward
  !
  ! 
$$\frac{F(j,n+1) - F(j,n)}{dt} + u \frac{F(j+1,n) - F(j,n)}{dx} = 0$$

  !
  INTEGER :: ok
  INTEGER :: j
  DO j=1,iMax
    PHI_P(j) = PHI_C(j) - (Uvel*Dt/Dx)*(PHI_C(j+1)-PHI_C(j))
  END DO
  CALL UpdateBoundaryLayer()
END FUNCTION SchemeForward

```

```

FUNCTION SchemeUpStream() RESULT(ok)
  IMPLICIT NONE
  ! Utilizando a diferenciacao forward no tempo e
  ! backward no espaco (upstream)
  !
  ! 
$$\frac{F(j,n+1) - F(j,n)}{dt} + u \frac{F(j,n) - F(j-1,n)}{dx} = 0$$

  !
  !
  INTEGER :: ok
  INTEGER :: j
  DO j=1,iMax
    PHI_P(j) = PHI_C(j) - (Uvel*Dt/Dx)*(PHI_C(j)-PHI_C(j-1))
  END DO
  CALL UpdateBoundaryLayer()
END FUNCTION SchemeUpStream

```

```

SUBROUTINE UpdateBoundaryLayer()
  IMPLICIT NONE
  PHI_P(0      ) = PHI_P(iMax )
  PHI_P(-1     ) = PHI_P(iMax-1)
  PHI_P(imax+1 ) = PHI_P(1)
  PHI_P(iMax+2 ) = PHI_P(2)
END SUBROUTINE UpdateBoundaryLayer

```

```

FUNCTION SchemeUpdate() RESULT (ok)
  IMPLICIT NONE
  INTEGER :: ok
  PHI_M=PHI_C
  PHI_C=PHI_P
  ok=0
END FUNCTION SchemeUpdate

```

END MODULE Class NumericalMethod

[illegible]



Métodos de diferenças finitas.

```

MODULE Class_WritetoGrads
USE Class_Fields, Only: PHI_A,PHI_P,PHI_C,PHI_M,Uvel,iMax
IMPLICIT NONE
PRIVATE
INTEGER, PUBLIC      , PARAMETER :: r8=8
INTEGER, PUBLIC      , PARAMETER :: r4=4
INTEGER              , PARAMETER :: UnitData=1
INTEGER              , PARAMETER :: UnitCtl=2
CHARACTER (LEN=400)   :: FileName
LOGICAL               :: CtrlWriteDataFile
PUBLIC :: SchemeWriteCtl
PUBLIC :: SchemeWriteData
PUBLIC :: InitClass_WritetoGrads
CONTAINS
SUBROUTINE InitClass_WritetoGrads()
  IMPLICIT NONE
  FileName=""
  FileName='AdvecLinearConceitual1D'
  CtrlWriteDataFile=.TRUE.
END SUBROUTINE InitClass_WritetoGrads

FUNCTION SchemeWriteData(irec) RESULT (ok)
  IMPLICIT NONE
  INTEGER , INTENT (INOUT) :: irec
  INTEGER               :: ok
  INTEGER               :: lrec
  REAL (KIND=r4)        :: Yout(iMax)
  INQUIRE (IOLENGTH=lrec) Yout
  IF(CtrlWriteDataFile)OPEN(UnitData,FILE=TRIM(FileName)//'.bin',&
  FORM='UNFORMATTED', ACCESS='DIRECT',
STATUS='UNKNOWN', &
  ACTION='WRITE',RECL=lrec)
  ok=1
  CtrlWriteDataFile=.FALSE.
  Yout=REAL(PHI_C(1:iMax),KIND=r4)
  irec=irec+1
  WRITE(UnitData,rec=irec)Yout

```

```

Yout=REAL(PHI_A(1:iMax),KIND=r4)
irec=irec+1
WRITE(UnitData,rec=irec)Yout
ok=0
END FUNCTION SchemeWriteData

FUNCTION SchemeWriteCtl(nrec) RESULT (ok)
IMPLICIT NONE
INTEGER, INTENT (IN) :: nrec
INTEGER :: ok
ok=1
OPEN(UnitCtl,FILE=TRIM(FileName)//'.ctl',FORM='FORMATTED', &
ACCESS='SEQUENTIAL',STATUS='UNKNOWN',ACTION='WRITE')
WRITE (UnitCtl,'(A6,A )')'dset ^',TRIM(FileName)//'.bin'
WRITE (UnitCtl,'(A )')'title EDO'
WRITE (UnitCtl,'(A )')'undef -9999.9'
WRITE (UnitCtl,'(A6,I8,A18 )')'xdef ',iMax,' linear 0.00 0.001'
WRITE (UnitCtl,'(A )')'ydef 1 linear -1.27 1'
WRITE (UnitCtl,'(A6,I6,A25 )')'tdef ',nrec,' linear 00z01jan0001 1hr'
WRITE (UnitCtl,'(A20 )')'zdef 1 levels 1000 '
WRITE (UnitCtl,'(A )')'vars 2'
WRITE (UnitCtl,'(A )')'phic 0 99 resultado da edol yc'
WRITE (UnitCtl,'(A )')'phia 0 99 solucao analitica ya'
WRITE (UnitCtl,'(A )')'endvars'
CLOSE (UnitCtl,STATUS='KEEP')
CLOSE (UnitData,STATUS='KEEP')
ok=0
END FUNCTION SchemeWriteCtl
END MODULE Class WritetoGrads

```



Métodos de diferenças finitas.

```
PROGRAM Main
USE Class_Fields, Only:Init_Class_Fields
USE Class_NumericalMethod, Only : InitNumericalScheme, &
    SchemeForward, SchemeUpdate, SchemeUpStream,&
    AnaliticFunction
USE Class_WritetoGrads, Only :InitClass_WritetoGrads, &
    SchemeWriteData, SchemeWriteCtl

IMPLICIT NONE
INTEGER           , PARAMETER :: r8=8
INTEGER           , PARAMETER :: r4=4
INTEGER           , PARAMETER :: xdim=100
REAL (KIND=r8)    , PARAMETER :: LX=1.0
REAL (KIND=r8)    , PARAMETER :: Uvel0=10.0!m/s
REAL (KIND=r8)    , PARAMETER :: dx=LX/xdim !m
REAL (KIND=r8)    , PARAMETER :: dt=0.1*dx/Uvel0 !s    !
c*Dt/Dx < 1
!                                     ! => Dt <
dx/Uvel0
INTEGER           , PARAMETER :: ninteraction=20000
CALL Init()
CALL run()
```

CONTAINS

```
SUBROUTINE Init()
IMPLICIT NONE
CALL Init_Class_Fields(xdim,Uvel0)
CALL InitNumericalScheme(dt,dx)
CALL InitClass_WritetoGrads
END SUBROUTINE Init
```

```
SUBROUTINE Run()
IMPLICIT NONE
INTEGER :: test,tn,irec
irec=0
DO tn=0,ninteraction
    test=SchemeUpStream()
    test=AnaliticFunction(tn)
    test=SchemeWriteData(irec)
    test=SchemeUpdate()
END DO
test=SchemeWriteCtl(ninteraction)
END SUBROUTINE Run

SUBROUTINE Finalize()
IMPLICIT NONE

END SUBROUTINE Finalize

END PROGRAM Main
```



Dedução da Velocidade de Fase Numérica

Ponto de Partida

A equação de advecção linear contínua é:

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0, \quad c = \text{const}$$

cuja solução exata é uma onda que se propaga com velocidade de fase **constante** c , sem distorção.



Métodos de diferenças finitas.



Passo 1: Discretização no Espaço

Substituímos a derivada espacial pela **diferença centrada de segunda ordem**:

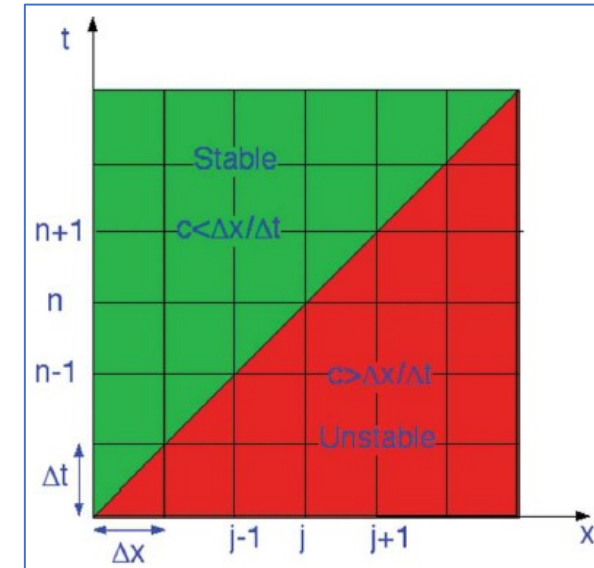
$$\frac{\partial u}{\partial x} \approx \frac{u_{j+1} - u_{j-1}}{2\Delta x}$$

obtendo a equação **diferencial-diferença**:

$$\frac{\partial u_j}{\partial t} = -c \frac{u_{j+1} - u_{j-1}}{2\Delta x}$$

O tempo ainda é **contínuo** — por isso é chamada de equação diferencial-diferença.

Qualquer erro aqui é **puramente da Discretização do espaço**.





Métodos de diferenças finitas.



Passo 2: Substituição da Solução Harmônica

Buscamos uma solução na forma de **componente harmônica simples**:

$$u_j(t) = \text{Re}[U(t) e^{ikj\Delta x}]$$

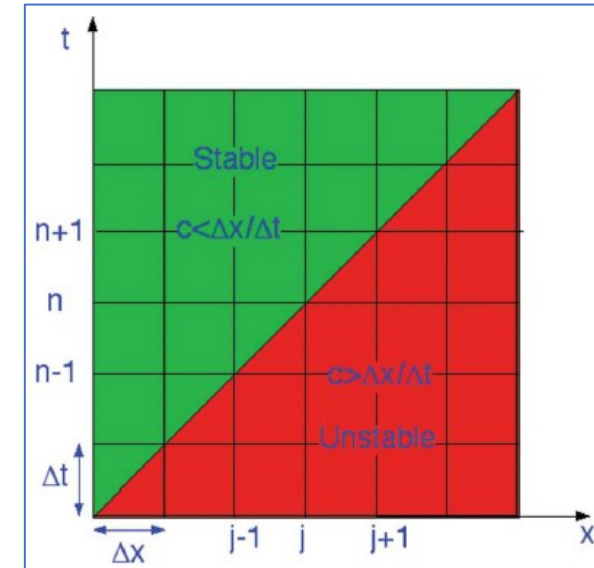
onde:

- k é o número de onda
- $U(t)$ é a amplitude (ainda função do tempo)
- $j\Delta x$ é a posição no grid

Calculamos $u_{j\pm 1}$:

$$u_{j+1} = \text{Re}[U(t) e^{ik(j+1)\Delta x}] = \text{Re}[U(t) e^{ikj\Delta x} e^{ik\Delta x}]$$

$$u_{j-1} = \text{Re}[U(t) e^{ik(j-1)\Delta x}] = \text{Re}[U(t) e^{ikj\Delta x} e^{-ik\Delta x}]$$





Métodos de diferenças finitas.

Passo 3: Diferença Centrada

Calculamos $u_{j+1} - u_{j-1}$:

$$u_{j+1} - u_{j-1} = \text{Re}[U(t) e^{ikj\Delta x} (e^{ik\Delta x} - e^{-ik\Delta x})]$$

Usando a identidade de Euler:

$$\text{sen } \theta = \frac{e^{i\theta} - e^{-i\theta}}{2i}$$

$$\sin(k\Delta x) = \frac{e^{ik\Delta x} - e^{-ik\Delta x}}{2i}$$

Portanto:

$$2i \sin(k\Delta x) = e^{ik\Delta x} - e^{-ik\Delta x}$$

$$u_{j+1} - u_{j-1} = \text{Re}[U(t) e^{ikj\Delta x} (2i \sin(k\Delta x))]$$



Métodos de diferenças finitas.

Passo 4: Substituição na Equação

Substituindo na equação diferencial-diferença:

$$\frac{\partial u_j}{\partial t} = -c \frac{u_{j+1} - u_{j-1}}{2\Delta x}$$

$$\frac{d}{dt} \text{Re} [U(t)e^{ikj\Delta x}] = -c \frac{\text{Re} [U(t)e^{ikj\Delta x} \cdot 2i \sin(k\Delta x)]}{2\Delta x}$$

Fatorando ($e^{ikj\Delta x}$) constante em t):

$$\frac{dU}{dt} = -c \frac{2i \sin(k\Delta x)}{2\Delta x} U$$

$$\boxed{\frac{dU}{dt} = -i \underbrace{\left(\frac{c}{\Delta x} \sin k \Delta x \right)}_{\omega^*} U}$$

$$u_j(t) = \text{Re}[U(t) e^{ikj\Delta x}]$$

$$\frac{\partial u_j(t)}{\partial t} = \frac{\partial \text{Re}[U(t)]}{\partial t} \text{Re}[e^{ikj\Delta x}] + \text{Re}[U(t)] \frac{\partial \text{Re}[e^{ikj\Delta x}]}{\partial t}$$

$$\frac{\partial u_j(t)}{\partial t} = \frac{\partial \text{Re}[U(t)]}{\partial t} \text{Re}[e^{ikj\Delta x}]$$

$$\frac{\partial u_j(t)}{\partial t} = \frac{\partial U(t)}{\partial t} \text{Re}[e^{ikj\Delta x}]$$



Métodos de diferenças finitas.

Passo 5: Identificação com a Equação de Oscilação

Esta equação tem exatamente a forma da **equação de oscilação** de ondas:

$$\frac{dU}{dt} = -i \underbrace{\left(\frac{c}{\Delta x} \sin k \Delta x \right)}_{\omega^*} U$$

$$\frac{dU}{dt} = i\omega U$$

com frequência **numérica**:

$$\omega^* = -\frac{c}{\Delta x} \sin(k\Delta x)$$

$$U(t) = U(0) e^{-i\omega^* t}$$

logo a solução completa é:

$$u_j(t) = \text{Re}[U(0) e^{i(kj\Delta x - \omega^* t)}]$$



Métodos de diferenças finitas.

Passo 6: Extração da Velocidade de Fase

Uma onda harmônica tem a forma geral:

$$u \sim e^{i(kx-vt)}$$

$$\omega^* = -\frac{c}{\Delta x} \sin(k\Delta x)$$

onde a velocidade de fase é definida como:

$$c = \frac{v}{k}$$

No nosso caso, comparando com $e^{i(kj\Delta x - \omega^*t)}$:

$$c^* = \frac{\omega^*}{k} = \frac{1}{k} \cdot \frac{c}{\Delta x} \sin(k\Delta x)$$

$$c^* = c \frac{\sin(k\Delta x)}{k\Delta x}$$



Métodos de diferenças finitas.

Passo 7: Análise do Resultado

Expandindo $\sin(k\Delta x)$ em série de Taylor:

$$c^* = c \frac{\sin(k\Delta x)}{k\Delta x}$$

$$\sin(k\Delta x) = k\Delta x - \frac{(k\Delta x)^3}{6} + \dots$$

Portanto:

$$c^* = c \left(k\Delta x - \frac{(k\Delta x)^3}{6} + \dots \right)$$

A diferenciação centrada causa **dispersão computacional**: ondas curtas se propagam mais devagar que o valor físico, e a onda mais curta resolvível ($2\Delta x$) fica **completamente estacionária** — o que é fisicamente errado. Esse é o erro de dispersão discutido no Cap. III, Seção 2 do Mesinger & Arakawa.

pela fórmula $k = \frac{2\pi}{\lambda}$.

$$\frac{c^*}{c} = \frac{\sin(k\Delta x)}{k\Delta x}$$

$$\frac{c^*}{c} = \left(k\Delta x - \frac{(k\Delta x)^3}{6} + \dots \right)$$

Comprimento de onda	$k\Delta x$	c^*/c
$\lambda \rightarrow \infty$ (ondas longas)	$k\Delta x = \frac{2\pi\Delta x}{\lambda}$ 0	1✓
$\lambda = 4\Delta x$	$k\Delta x = \frac{2\pi\Delta x}{\lambda}$ $\pi/2$	$2/\pi \approx 0.64$
$\lambda = 2\Delta x$ (mínima)	$k\Delta x = \frac{2\pi\Delta x}{\lambda}$ π	0 — estacionária!

Conclusão Física

$c^* < c$ para todas as ondas curtas $c^* = 0$ para a onda de $2\Delta x$



Dispersão (Dispersion)

Ocorre quando **diferentes comprimentos de onda se propagam com velocidades de fase diferentes** do valor verdadeiro. O esquema não amortece as ondas — ele as **distorce no espaço**.

Arakawa: a diferenciação centrada de segunda ordem produz uma velocidade de fase numérica

$$c^* = c \frac{\sin k \Delta x}{k \Delta x}$$

que é **menor** que a velocidade real c para ondas curtas.

As ondas ainda se propagam, mas na velocidade errada. Isso deforma padrões — frentes, linhas de cisalhamento — que são superposição de muitos comprimentos de onda.

Analogia: imagine uma orquestra onde cada instrumento toca na velocidade certa, mas chega ao ouvinte em tempos diferentes. A música (o padrão) fica distorcida.



Dissipação (Dissipation)

Ocorre quando o esquema **amortece a amplitude das ondas** com o tempo.
A energia é artificialmente removida da solução.

No livro: o esquema de Lax-Wendroff tem fator de amplificação $|\lambda| < 1$ para ondas curtas — elas **decaem** numericamente mesmo que a equação verdadeira as conserve.

Analogia: a orquestra toca no ritmo certo, mas alguns instrumentos vão ficando mais baixos até sumir.



Comparação direta

Propriedade	Dispersão	Dissipação
Amplitude	Conservada	Reduzida
Velocidade de fase	Errada	Pode ser correta
Efeito visual	Distorção do padrão	Suavização / amortecimento
Esquema típico	Leapfrog + dif. centrada	Lax-Wendroff, upstream
\$	λ	\$
Erro principal	Fase ($\theta/p \neq 1$)	Amplitude (\$

Um esquema pode ter ambos:

O esquema **upstream** (diferenciação a montante), por exemplo:

- É **dissipativo** — amortece ondas curtas fortemente
- Tem pouca dispersão espúria — não gera ondas parasitas

O esquema **leapfrog com diferenciação centrada**:

- É **neutro** ($|\lambda| = 1$) — não dissipa
- Mas é **dispersivo** — velocidade de fase errada para ondas curtas



Por que importa na meteorologia?

Arakawa: destaca que a **dispersão computacional** é especialmente problemática para padrões de pequena escala como frentes atmosféricas, que contêm proporção significativa de ondas curtas.

Esses padrões se deformam rapidamente durante a integração numérica.

Já a **dissipação** pode ser útil ou prejudicial: prejudicial nos termos de advecção (remove energia fisicamente), mas útil para controlar a **instabilidade não-linear** — daí o uso intermitente do esquema de Lax-Wendroff na prática operacional descrito por Arakawa.



Métodos de diferenças finitas.

Esquemas com diferenças centradas no espaço de quarta ordem:

$$\frac{u_{j+1} - u_{j-1}}{2\Delta x} = \frac{\partial u}{\partial x} + \frac{1}{3!} \frac{\partial^3 u}{\partial x^3} (\Delta x)^2 + O[(\Delta x)^4]$$

O Quociente é de segunda ordem de acurácia. É formado considerando a diferença de valores de u_j em um ponto de grade distante do ponto central. Simularmente um quociente pode ser construído considerando a diferença de dois pontos distante do ponto central. Substituindo Δx por $2\Delta x$

$$\frac{u_{j+2} - u_{j-2}}{4\Delta x} = \frac{\partial u}{\partial x} + \frac{4}{3!} \frac{\partial^3 u}{\partial x^3} (\Delta x)^2 + O[(\Delta x)^4]$$

O Quociente é de segunda ordem de acurácia. Mas os coeficientes são grandes. Outra aproximação consistente para a derivada espacial pode ser construída pela combinação linear das duas equações acima. A combinação para o termo de segunda ordem no erro de truncamento é cancelada

$$\frac{4}{3} \frac{u_{j+1} - u_{j-1}}{2\Delta x} - \frac{1}{3} \frac{u_{j+2} - u_{j-2}}{4\Delta x} = \frac{\partial u}{\partial x} + O[(\Delta x)^4]$$



Métodos de diferenças finitas.

Esquemas com diferenças centradas no espaço de quarta ordem:

$$\frac{\partial u}{\partial x} = \frac{4}{3} \frac{u_{j+1} - u_{j-1}}{2\Delta x} - \frac{1}{3} \frac{u_{j+2} - u_{j-2}}{4\Delta x} - O[(\Delta x)^4]$$

Por Exemplo: Resolvendo a equação de advecção linear

$$\frac{\partial u}{\partial t} = -c \frac{\partial u}{\partial x}$$

Na solução analítica o estado inicial de propaga no espaço com velocidade de fase constante c sem mudança de forma.

A solução numérica, entretanto, **ficará atrasada em relação a analítica** e **dispersa**.

A alta ordem de acurácia o atraso será menor (a diferença de fase será menor).

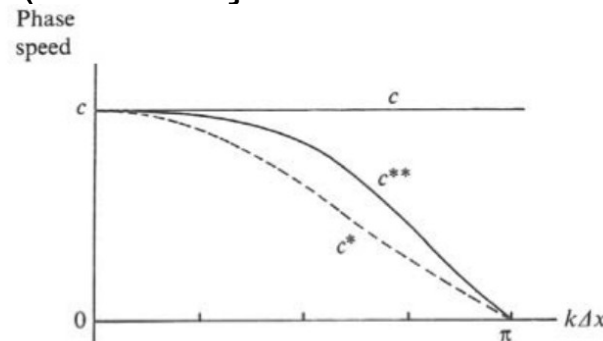
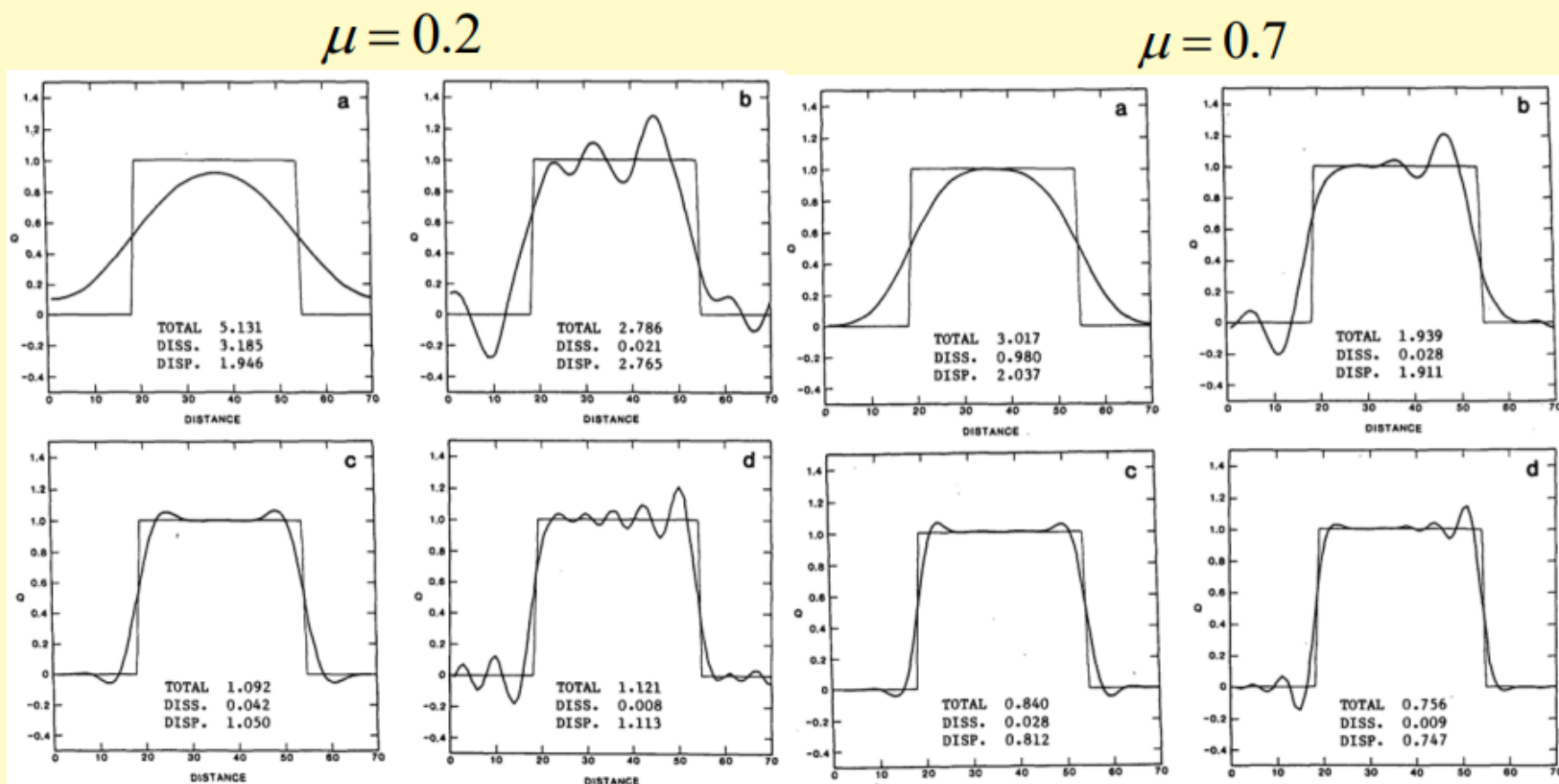


Figure 4.1 Phase speed for the linear advection equation, c , and for the corresponding differential-difference equations with second order (c^*) and with fourth order (c^{**}) centered space differencing.



Métodos de diferenças finitas.

O Aumento da ordem de acurácia não resolve o problema de dispersão que envolve a solução numérica (Takacs,1985)



Numerical solutions for (a) 1st-order scheme, (b) 2nd-order scheme, (c) 3rd-order scheme, and (d) 4th-order scheme, after two complete translations



Exercício 1

Resolva numericamente a equação de advecção utilizando a acurácia de quarta ordem na discretização espacial e compare com a solução de segunda ordem. Considere $\mu = \frac{c\Delta x}{\Delta t} = 0.2$ e $\mu = \frac{c\Delta x}{\Delta t} = 0.7$. Como condição inicial use uma onda quadrada.

$$\frac{\partial u}{\partial x} = \frac{4}{3} \frac{u_{j+1} - u_{j-1}}{2\Delta x} - \frac{1}{3} \frac{u_{j+2} - u_{j-2}}{4\Delta x} - O[(\Delta x)^4]$$

$$\frac{\partial u}{\partial t} = -c \frac{\partial u}{\partial x}$$

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = -c \left(\frac{4}{3} \frac{u_{j+1} - u_{j-1}}{2\Delta x} - \frac{1}{3} \frac{u_{j+2} - u_{j-2}}{4\Delta x} \right)$$

$$u_j^{n+1} = u_j^n - c \frac{\Delta t}{\Delta x} \left(\frac{4}{3} \frac{u_{j+1} - u_{j-1}}{2\Delta x} - \frac{1}{3} \frac{u_{j+2} - u_{j-2}}{4\Delta x} \right)$$

$$u_j^{n+1} = u_j^n - c \frac{\Delta t}{\Delta x} \left(\frac{4}{3} \frac{u_{j+1} - u_{j-1}}{2} - \frac{1}{3} \frac{u_{j+2} - u_{j-2}}{4} \right)$$

$$u_j^{n+1} = u_j^n - c \frac{\Delta t}{\Delta x} \left(\frac{4u_{j+1} - 4u_{j-1}}{6} - \frac{u_{j+2} - u_{j-2}}{12} \right)$$

$$u_j^{n+1} = u_j^n - c \frac{\Delta t}{\Delta x} \left(\frac{8u_{j+1} - 8u_{j-1} - u_{j+2} + u_{j-2}}{12} \right)$$

$$u_j^{n+1} = u_j^n - c \frac{\Delta t}{\Delta x} \left(\frac{-u_{j+2} + 8u_{j+1} - 8u_{j-1} + u_{j-2}}{12} \right)$$

$$u_j^{n+1} = u_j^n - \mu \left(\frac{-u_{j+2} + 8u_{j+1} - 8u_{j-1} + u_{j-2}}{12} \right)$$